

VX/TPU — Engineering Reference Manual

Sector7 USA LLC · EDIT/TPU for Linux and Windows

Engineering reference, July 2026 · V1.08-0

Table of contents

- [VX/TPU - Text Processing Utility Reference Manual](#)
 - [1. Overview](#)
 - [1.1 What TPU is](#)
 - [1.2 How TPU differs from EDT](#)
 - [1.3 What EVE is](#)
 - [1.4 The Sector7 model](#)
 - [2. Invocation](#)
 - [2.1 DCL form \(VMS heritage\)](#)
 - [2.2 Host shell form \(the real entry point\)](#)
 - [2.3 Environment variables](#)
 - [2.4 Exit codes](#)
 - [3. The TPU Language](#)
 - [3.1 Lexical structure](#)
 - [3.2 Statements](#)
 - [3.3 Data types](#)
 - [3.4 Operators](#)
 - [3.5 Variable scope](#)
 - [3.6 Compile / interpret pipeline](#)
 - [4. Built-In Procedures \(the API\)](#)
 - [4.1 Cursor / motion](#)
 - [4.2 Text editing](#)
 - [4.3 Mark / selection / range](#)
 - [4.4 Search / match](#)
 - [4.5 Buffer / file management](#)
 - [4.6 Windows](#)
 - [4.7 Keymap and key binding](#)
 - [4.8 SET \(attribute\) - the universal setter](#)
 - [4.9 Status / introspection](#)
 - [4.10 GET_INFO](#)
 - [4.11 Pattern construction](#)
 - [4.12 Messages and miscellaneous](#)
 - [4.13 Procedure / control / exit](#)
 - [4.14 String / numeric utility \(expression-only\)](#)
 - [4.15 Reading user input](#)
 - [4.16 Test harness](#)
 - [4.17 LEARN](#)
 - [4.18 Journal](#)
 - [4.19 External processes](#)
 - [4.20 Arrays](#)
 - [4.21 COMPILE / PROGRAM](#)
 - [4.22 Clipboard](#)
 - [4.23 File-name helpers](#)

- [4.24 DECwindows shim \(accepted-no-op\)](#)
- [4.25 CALL_USER](#)
- [5. Built-in Constants, Keywords, and System Variables](#)
 - [5.1 Direction / mode keywords](#)
 - [5.2 Pattern keywords](#)
 - [5.3 Position keywords](#)
 - [5.4 System buffers](#)
 - [5.5 System variables](#)
 - [5.6 SET attributes - complete reference](#)
 - [5.7 FAO directives](#)
- [6. Buffer / Window / Range / Marker Model](#)
 - [6.1 Buffers](#)
 - [6.2 Windows](#)
 - [6.3 Markers](#)
 - [6.4 Ranges](#)
 - [6.5 Pattern matching engine](#)
- [7. Section Files, Command Files, and Initialisation](#)
 - [7.1 Portable section files \(#VXSECT\)](#)
 - [7.2 Command and initialisation files](#)
 - [7.3 The TPU\\$INIT_PROCEDURE hook](#)
- [8. EVE on top of TPU](#)
 - [8.1 EVE command vocabulary](#)
 - [8.2 Kill ring and named pastes](#)
 - [8.3 EDT\\$EMULATE](#)
- [9. Journal / Recovery](#)
 - [9.1 Format](#)
 - [9.2 Opening and closing](#)
 - [9.3 Recovery](#)
- [10. Keypad Model](#)
 - [10.1 The default key map](#)
 - [10.2 Key names](#)
 - [10.3 GOLD prefix lifecycle](#)
 - [10.4 Custom bindings](#)
- [11. Pattern Language](#)
 - [11.1 BNF sketch](#)
 - [11.2 Operator semantics](#)
 - [11.3 Examples](#)
- [12. Compile Model - How tpu.cpp Parses and Runs](#)
 - [12.1 The interpreter is hand-rolled](#)
 - [12.2 The pipeline](#)
 - [12.3 Lexer / scanner](#)
 - [12.4 Parser](#)
 - [12.5 Expression evaluator](#)
 - [12.6 Built-in dispatcher](#)
 - [12.7 Memory model](#)

- [12.8 Top-level C++ layout](#)
- [13. Known Divergences from VAX/VMS TPU](#)
 - [13.1 Policy-deferred \(not in the close loop\)](#)
 - [13.1b Closed formerly-deferred items](#)
 - [13.2 Behavioural notes \(host / subset\)](#)
 - [13.2b Arrays \(DEC surface implemented\)](#)
 - [13.3 Partial forms still differing from full DEC](#)
 - [13.4 Sector7 additions DEC TPU does not have](#)
- [14. File Reference](#)
 - [14.1 The source tree](#)
 - [14.2 Key entry points](#)
 - [14.3 Test harness](#)
 - [14.4 Build](#)
 - [14.5 Compile-time options](#)
- [15. End Notes](#)
 - [15.1 Copyright](#)
 - [15.2 Build provenance](#)
 - [15.3 Authorship and contact](#)
 - [15.4 Document history](#)

VX/TPU - Text Processing Utility Reference Manual

Sector7 USA LLC - Standalone C++ port of DEC TPU

Implementation reference for `tpu.cpp` (~8,000 lines), the single translation unit at `/Users/jonpower/s7tools/vxtools85/tpu/tpu.cpp` that delivers a faithful subset of DEC's Text Processing Utility (DECTPU), the engine behind the EVE editor on OpenVMS, plus an EVE compatibility layer that implements the keypad bindings, command-line vocabulary, and `SET` attributes a VAX/VMS user expects.

Release: V1.08-0 (2026-07-24). Compared against OpenVMS DECTPU

```
GET_INFO(SYSTEM, "version")=3 / "update"=2 on host t86.
```

This document is the engineering-level reference: every built-in procedure the interpreter actually implements, every command keyword the line-mode prompt accepts, every default key binding, every `SET ( . . . )` attribute, the pattern engine, the language grammar, the interpreter pipeline, and the catalogue of features that are stubs, accepted no-ops, or policy- deferred. Where behaviour differs from VAX/VMS TPU it is called out in section 13. See also `TPU_CHANGELOG.md` for the release history.

1. Overview

1.1 What TPU is

TPU - the Text Processing Utility - is a programmable text editor that DEC shipped with VMS. Users normally invoke it with the DCL command

```
$ EDIT/TPU foo.txt
```

which loads the standard section file containing the EVE editor and opens `foo.txt` for editing. EVE is the user interface; TPU is the substrate beneath it - a Pascal-flavoured interpreted language with first-class values for buffers, ranges, markers, windows, patterns, key maps, and external processes, and a few hundred built-in procedures that manipulate them.

A TPU **session** consists of:

- one or more **buffers** (text + cursor + flags + undo state),
- zero or more **windows** (viewports onto buffers),
- zero or more **key maps** organised into **key map lists** that decide what each keystroke does,
- a global pool of **variables** and **user procedures**, and
- a current state of the **interpreter** (control-flow signal, the current procedure scope frames, the error-handler stack).

1.2 How TPU differs from EDT

EDT is a line/character editor with a fixed command vocabulary. TPU is a **programmable** editor: every keystroke runs an expression in the TPU language, the standard keypad is defined by user-replaceable bindings, and procedures can compose buffers, ranges, and patterns into arbitrary editing tasks. Where EDT has commands like `SUBSTITUTE` and `EXIT`, TPU has Built-In Procedures (`SUBSTITUTE`, `EXIT`, ... callable from any script) and the EVE keymap that calls them.

EDT-style line-mode editing is available inside EVE via `EDT$EMULATE` on OpenVMS; the VX/TPU port does not bundle that emulation but it does provide a single-line command prompt (`Ctrl-Z` brings up `*`) that resembles EVE's command line and accepts the EVE vocabulary.

1.3 What EVE is

EVE is the section file delivered with VMS that turns raw TPU into the familiar editor: the LK201 keypad bindings, the `WRITE FILE`, `EXIT`, `GET file`, `INCLUDE file`, `FIND`, `BUFFER`, `SHOW BUFFERS`, etc. command vocabulary, the buffer ring with `NEXT BUFFER/ PREVIOUS BUFFER`, the kill ring, the wrap/margin/tab support, and a hundred-odd small conveniences a user expects of a real editor. On a stock VMS install, `SYSSHARE:EVE$SECTION.TPU$SECTION` is loaded automatically.

In the Sector7 port the EVE personality is **hard-wired** into the C++ binary - there is no separate `.TPU$SECTION` file. The default key map called `TPU$KEY_MAP` is installed at start-up (see `define_default_keys` at `tpu.cpp:3040`); the EVE command-line vocabulary is implemented in `execute_eve_command` (`tpu.cpp:3605`).

1.4 The Sector7 model

The Sector7 build (`vxtools85/tpu/`) is a self-contained C++17 program:

- **Single source file** - `tpu.cpp` (303 KB, 7,282 lines).
- **Single binary** - `tpu.exe` (about 1.2 MB on macOS).
- **Curses-based** - ncurses on POSIX, PDCurses on Windows.
- **No `.TPU$SECTION` byte-code** - the editor is the section file.
- **No `.TPU$JOURNAL` recovery format** - the `JOURNAL_OPEN`, `JOURNAL_CLOSE`, `RECOVER_BUFFER` built-ins write/replay a plain TPU script (one statement per line).
- **No callable interface** - DECTPU's `TPU$INITIALIZE` / `TPU$EXECUTE_COMMAND` callable interface is not exposed; everything runs from `main()` in interactive or `--test` mode.

The major C++ classes (top to bottom in `tpu.cpp`):

Class / struct	File range	Role
<code>Marker</code>	<code>tpu.cpp:188-198</code>	(buffer, row, col, free_cursor) triple
<code>Range</code>	<code>tpu.cpp:205-274</code>	Half-open span [begin, end) within one buffer
<code>TPUPatternToken</code>	<code>tpu.cpp:276-309</code>	One operator in a DECTPU pattern
<code>TPUPattern</code>	<code>tpu.cpp:315-529</code>	Sequence of tokens; matcher
<code>Buffer</code>	<code>tpu.cpp:531-1582</code>	Lines, cursor, flags, undo, video regions
<code>TPUProcess</code>	<code>tpu.cpp:1583-1785</code>	External process with pipe
<code>TPUWindow</code>	<code>tpu.cpp:1786-1814</code>	Viewport: geometry + buffer pointer
<code>KeyBinding</code>	<code>tpu.cpp:1815-1822</code>	(procedure text, native fn, help)
<code>KeyMap</code>	<code>tpu.cpp:1823-1851</code>	int-keycode -> KeyBinding
<code>KeyMapList</code>	<code>tpu.cpp:1852-1907</code>	Ordered list of KeyMaps
<code>Value</code>	<code>tpu.cpp:1915-1994</code>	Dynamically-typed TPU value
<code>Stmt</code>	<code>tpu.cpp:2005-2014</code>	Compiled statement (Kind + payload)
<code>Procedure</code>	<code>tpu.cpp:2015-2024</code>	name + params + body
<code>MiniTPUParser</code>	<code>tpu.cpp:2073-2614</code>	Lexer + parser + interpreter
<code>TPUEditor</code>	<code>tpu.cpp:2645-...</code>	Screen, run loop, command line

Class / struct	File range	Role
splash + vx_redirect_tty	tpu.cpp:7000-7211	VX_TTY redirection, splash screen
main	tpu.cpp:7212-7282	CLI entry, --test, startup files

2. Invocation

2.1 DCL form (VMS heritage)

On VAX/VMS the canonical invocation is

```
$ EDIT/TPU [/qualifier ...] [file]
```

The VX/TPU port accepts both Unix-style argv and a subset of DCL-style slash qualifiers (`/COMMAND=`, `/SECTION=`, `/NOSECTION`, and the `--command` / `--section` spellings). Qualifiers not listed as supported below remain documentation-only heritage.

DCL qualifier	VMS meaning	VX/TPU equivalent
<code>/SECTION=file</code>	Pre-compiled section file (byte-code) loaded at start	Loads portable <code>#VXSECT</code> section (<code>RESTORE</code>)
<code>/COMMAND=file</code>	Text file of TPU commands sourced before init	Compiled after <code>\$TPU_STARTUP</code> / <code>~/ .tpurc</code>
<code>/INITIAL=file</code>	Alternative init file (instead of the standard)	Same as <code>/COMMAND=</code>
<code>/NOSECTION</code>	Suppress loading any section	Accepted (no-op beside clearing a pending <code>/SECTION=</code>); bindings remain active
<code>/JOURNAL=name</code>	Override journal-file name	<code>JOURNAL_OPEN(path)</code> built-in (per buffer)
<code>/NOJOURNAL</code>	Disable journaling	<code>JOURNAL_CLOSE</code> built-in or <code>SET (JOURNALING, OFF)</code>
<code>/RECOVER</code>	Replay the journal into the buffer at start	<code>RECOVER_BUFFER(path)</code> built-in
<code>/OUTPUT=file</code>	Write to alternate filename on exit	<code>WRITE_FILE(buffer, "path")</code> (DEC form) or host <code>WRITE_FILE("path")</code>
<code>/READ_ONLY</code>	Open file read-only	<code>SET (NO_WRITE, ON)</code> after loading
<code>/DEBUG[=facility]</code>	Compile-time debug logging	Not implemented
<code>/DISPLAY=DECWINDOWS</code>	DECwindows display	<code>DEFINE_WIDGET_CLASS</code> etc. are accepted as no-ops (<code>tpu.cpp:5994</code>)
<code>/INTERFACE=CHARACTER_CELL</code>	Character-cell mode	This is the only mode
<code>/MODIFY</code>	Allow write back	Default

DCL qualifier	VMS meaning	VX/TPU equivalent
/CREATE	Create file if absent	Default - if the file does not exist <code>load_file</code> returns false and a fresh buffer is created
/WORK=file	Override work file	Not implemented
/START_POSITION=(l,c)	Open at line l, column c	Not parsed; use <code>GO TO <line></code> after start

2.2 Host shell form (the real entry point)

```
tpu [options] [file]
```

Parsed by `main()`:

Argv form	Behaviour
<code>tpu</code>	Open with an empty buffer named <code>MAIN</code>
<code>tpu file.txt</code>	Open <code>file.txt</code> in buffer <code>file.txt</code> (the buffer's <code>name</code> is the path)
<code>tpu --test path.tpu</code>	Headless test runner: compile and execute <code>path.tpu</code> , emit <code>PASS/FAIL</code> log, exit 0/1
<code>tpu /COMMAND=file</code>	Compile <code>file</code> after startup scripts (also <code>--command=</code>)
<code>tpu /SECTION=file</code>	Accepted; prints a deferred-section warning (also <code>--section=</code>)

`--test` is exclusive and takes the script path as the next argument. Other args may mix a source file with `/COMMAND=` / `/SECTION=` forms.

2.3 Environment variables

Sampled in `main()`, `vx_redirect_tty()`, and the splash screen:

Variable	Honoured at	Effect
<code>S7SECURITY</code>	<code>main()</code> (optional)	Full path to the settings/splash file shared with EDT. If unset, <code>\$HOME/S7SECURITY.INI</code> (Windows: <code>%USERPROFILE%\S7SECURITY.INI</code>). Optional — missing/unreadable is fine, TPU runs regardless; the file is auto-created carrying <code>splash=off</code> after the one-time splash.
<code>TPU_STARTUP</code>	<code>main()</code>	Path to a TPU script auto-compiled before the run loop. Overrides <code>~/ .tpurc</code> .
<code>HOME</code>	<code>main()</code>	If <code>TPU_STARTUP</code> is unset, <code>\$HOME/ .tpurc</code> is sourced when present; also default license path
<code>USERPROFILE</code>	<code>main()</code> (Windows)	Fallback for <code>HOME</code>

Variable	Honoured at	Effect
<code>TPU_NO_SPLASH</code>	<code>splash</code>	Suppresses the start-up splash for a single run without persisting <code>splash=off</code>
<code>VX_TTY</code>	<code>vx_redirect_tty</code>	Re-binds stdin/stdout/stderr to the named TTY device (e.g. <code>/dev/ttys123</code>)
<code>TERM</code>	<code>curses</code>	Honoured by ncurses; default is <code>xterm-256color</code>

Optional splash / `S7SECURITY.INI`: the file is **no longer required** — TPU runs whether or not it exists. It controls a one-time branded splash. On first launch (no suppression flag yet) the splash is shown once with **no timeout** — it stays up until the user hits ENTER, and says on screen that it is shown once only — after which TPU writes a plain-text `S7SECURITY.INI` containing `splash=0` at the resolved path so it is skipped on subsequent launches.

To see the splash again, set `splash=on` (once, waits for ENTER) or `splash=<n>` for a timed display of `n` seconds, clamped to 0..15 (anything outside that range means 15); ENTER dismisses a timed splash early, other keys are ignored. `splash=0` is the same as `splash=off`. An unrecognised value is ignored with a warning on stderr naming it. Either way the splash reverts to `0` after being shown.

A legacy encrypted `S7SEC1` blob is still read tolerantly to supply the `(c) Sector7 USA LLC TPU licensed to <name> <email>` splash line, but is not required and is single-use (replaced by the plain flag once the splash shows). Shared sources live under `editors/s7security/` (`mksecurity` / `s7licdb` create the encrypted customer blobs).

`VX_TTY` is the bridge that lets a detached VX/RT server pop up an editor on an existing PTY without spawning a fresh shell.

2.4 Exit codes

Code	Meaning
0	Normal exit (<code>EXIT</code> / <code>QUIT</code> / Ctrl-X)
1	One or more <code>ASSERT*</code> failures during <code>--test</code>
2	<code>--test</code> could not open the script (the security file is optional and never causes an exit)
n	The user passed <code>EXIT(n)</code> or <code>QUIT(n)</code> to set an explicit exit code

The explicit form is set at `tpu.cpp:5521-5526`:

```
if (name == "QUIT" || name == "EXIT") {
    auto a = eval_args();
    if (!a.empty()) editor.exit_code = static_cast<int>(a[0].as_int());
    editor.running = false;
```

```
    return;  
}
```

3. The TPU Language

3.1 Lexical structure

The interpreter is **not** built from a classical token stream. Instead the source text is preprocessed in three passes (`tpu.cpp:2177-2289`), then statements are recognised on the way into `parse_block`:

1. `strip_comments` (`tpu.cpp:2177`) - replaces `! ... newline` tail comments with a bare newline so the source-line counter stays in sync.
2. `join_continuations` (`tpu.cpp:2197`) - folds a trailing dash (`-`) followed by optional whitespace and a newline into a single logical line.
3. `split_statements` (`tpu.cpp:2220`) - cuts the text into a `std::vector<std::string>` at `;` and `\n` boundaries, respecting double-quoted strings.

3.1.1 Identifiers

Identifiers start with a letter or underscore, followed by letters, digits, or underscores. Case is **not significant**: every identifier is upper-cased via the `upper()` helper before lookup. Built-in keywords are spelled in upper case in this document by convention.

Reserved-name patterns recognised by the dispatcher: see section 5.

3.1.2 Integer literals

Integer literals are accepted in four radices (`tpu.cpp:6420-6450`):

Prefix	Example	Meaning
(none)	<code>42, -3</code>	Decimal
<code>%D</code>	<code>%D42</code>	Decimal (explicit)
<code>%X</code> or <code>%H</code>	<code>%X1A, %HFF</code>	Hexadecimal
<code>%O</code>	<code>%O17</code>	Octal
<code>%B</code>	<code>%B1010</code>	Binary

```
ASSERT_EQ(%X1A, 26, "%X1A hex");
ASSERT_EQ(%XFF, 255, "%XFF hex");
ASSERT_EQ(%O17, 15, "%O17 octal");
ASSERT_EQ(%B1010, 10, "%B1010 binary");
ASSERT_EQ(%D42, 42, "%D42 decimal-with-prefix");
```

(from `tests/tpu_tests.tpu` lines 175-179.)

3.1.3 String literals

TPU strings are delimited by **double quotes** (`"hello"`). Single quotes are not string delimiters. Strings do not support backslash escapes; embedded double quotes are written as `""` (DEC convention). See `strip_quotes` (`tpu.cpp:2597`).

3.1.4 Keyword (symbolic) values

Identifiers that do not resolve to a variable or procedure name are returned as `Value::KEYWORD` (`tpu.cpp:6998`). This is how the `FORWARD`, `REVERSE`, `OFF`, `ON`, `UPPER`, `LOWER` etc. arguments to `SET`, `EDIT`, and `SEARCH` work - they are just unresolved identifiers that the receiving built-in inspects.

3.1.5 Comments

`!` to end of line. Comments can contain anything except a literal newline; they do not nest. See `strip_comments` (`tpu.cpp:2177`).

3.1.6 Line continuation

A trailing `-` immediately before a newline (with optional intervening whitespace) is consumed; the next line is concatenated. See `join_continuations` (`tpu.cpp:2197`).

```
v := 1 + -  
    2 + -  
    3;  
! v = 6
```

3.2 Statements

The Stmt tree (see `tpu.cpp:2005-2014`) has these kinds:

- `CALL` - dispatch a built-in or user procedure.
- `ASSIGN` - `lhs := rhs`. The LHS may be `array{key}`.
- `IF` - condition + body + else_body (see `parse_block`, `tpu.cpp:2358`).
- `LOOP` - body (see `parse_block`, `tpu.cpp:2430`).
- `EXITIF` - conditional break out of the innermost LOOP.
- `RETURN` - exit current procedure.
- `ABORT_` - raise the abort signal (caught by `ON_ERROR`).
- `BREAK_` - unconditional loop exit.
- `ON_ERROR_` - declarative handler; not executed unless the enclosing block aborts.

3.2.1 Assignment

```
x := 1;
msg := "hello " + CURRENT_BUFFER;
a{"key"} := 42;
```

Assignment uses `:=`. The right-hand side is evaluated by `eval_expression`. The left-hand side may be a bare name (case insensitive) or an array subscript (`name{key}`).

3.2.2 LOCAL / CONSTANT / VARIABLE declarations

```
LOCAL a, b := 0, c := "x";
```

`LOCAL`, `CONSTANT`, and `VARIABLE` are recognised at the start of a statement and produce zero or more `ASSIGN` statements (one per initialiser). Bare names declare nothing - TPU variables are dynamically created on first assignment. See `parse_block` (`tpu.cpp:2309-2346`).

The `CONSTANT` keyword is enforced: the first assignment binds the name; later reassignment records `TPU$_CONTRADEF` and leaves the value unchanged.

3.2.3 IF / ELSE / ENDIF

```
IF cond THEN
    ...
ELSE
    ...
ENDIF;
```

Both `THEN` body and `ELSE` body may be:

- A series of statements on subsequent lines (until `ELSE` or `ENDIF`).
- A single inline statement on the same line as `THEN` / `ELSE`.

Inline forms are tolerated:

```
IF 1 = 1 THEN flag := 1; ELSE flag := 2; ENDIF;
```

See `parse_block` (`tpu.cpp:2358-2428`).

3.2.4 LOOP / EXITIF / ENDLOOP

```
LOOP
    EXITIF cond;
```

```
...
ENDLOOP;
```

`LOOP ... ENDLOOP` is the only loop construct. Exit by `EXITIF`, `BREAK`, `RETURN`, or `ABORT`. `BREAK` is unconditional and exits one level; `EXITIF` evaluates a condition every iteration. See `parse_block` (`tpu.cpp:2430-2438`).

3.2.5 PROCEDURE / ENDPROCEDURE

```
PROCEDURE my_proc(a, b)
    LOCAL r := a + b;
    MESSAGE(STR(r));
ENDPROCEDURE;
```

Parameters are bound by position. Each call pushes a fresh scope frame (`local_scopes.emplace_back()`); the frame is popped on return regardless of the exit cause. See `execute_procedure_with_args` (`tpu.cpp:2571`).

If the procedure body raises `RETURN`, `ABORT`, or a runtime error, the frame is unwound and control returns to the caller. `ABORT` propagates upward until an `ON_ERROR` handler is seen.

3.2.6 ON_ERROR

```
PROCEDURE will_fail
    ON_ERROR
        recovered := 1;
    ENDON_ERROR;
    ABORT;
ENDPROCEDURE;
```

`ON_ERROR ... ENDON_ERROR` declares a handler at any point in a block; when the block aborts or signals `RAISE_`, the handler runs in the **same** scope frame. See `execute_block` (`tpu.cpp:6030`).

`ON_ERROR` handlers do not nest within a block (only one is recognised per block); they do nest by procedure scope.

3.3 Data types

TPU values are tagged unions; the C++ type is `Value` (see `tpu.cpp:1915`). Kinds:

Kind	Payload field	Examples
INTEGER	<code>i: long long</code>	<code>42</code> , <code>LENGTH("foo")</code>
STRING	<code>s: std::string</code>	<code>"hello"</code> , <code>CURRENT_LINE</code>

Kind	Payload field	Examples
KEYWORD	<code>s: std::string</code>	<code>FORWARD</code> , <code>UPPER</code>
PATTERN	<code>pattern: shared_ptr</code>	<code>ANCHOR + "x"</code> , <code>ANY("abc")</code>
RANGE	<code>range: shared_ptr</code>	<code>SEARCH(...)</code> , <code>CREATE_RANGE(...)</code>
MARKER	<code>marker: shared_ptr</code>	<code>MARK</code> , <code>BEGINNING_OF(buffer)</code>
BUFFER	<code>buffer: Buffer*</code>	<code>CURRENT_BUFFER</code> , <code>BUFFER("name")</code>
ARRAY	<code>array: shared_ptr</code>	<code>CREATE_ARRAY</code>
PROGRAM	<code>program: shared_ptr</code>	<code>COMPILE("MESSAGE(\"hi\")");</code>
LEARN	<code>learn_seq: shared_ptr</code>	<code>LEARN_END</code>
PROCESS	<code>process: shared_ptr</code>	<code>CREATE_PROCESS("ls", buf)</code>
NONE	(nothing)	Default-constructed, missing variable

Implicit coercion (`Value::as_int`, `as_string`, `truthy`):

- `STRING -> INTEGER` via `std::stoll` (errors yield 0).
- `INTEGER -> STRING` via `std::to_string`.
- `BUFFER -> STRING` returns the buffer name.
- `RANGE -> STRING` returns the text of the range.
- `MARKER -> STRING` returns `<marker buf:row,col>`.
- `BOOL` projection: `INTEGER` truthy iff non-zero; `STRING` truthy iff non-empty; collection types truthy iff non-empty/non-null.

3.4 Operators

Layer	Operators	Source
OR	<code>OR</code>	<code>eval_or</code>
AND	<code>AND</code>	<code>eval_and</code>
Comparison	<code>=</code> , <code><></code> , <code><</code> , <code>></code> , <code><=</code> , <code>>=</code>	<code>eval_compare</code>
Additive	<code>+</code> , <code>-</code> , <code>&</code>	<code>eval_add</code>
Multiplicative	<code>*</code> , <code>/</code>	<code>eval_mul</code>
Unary	<code>NOT</code> , unary <code>+</code> / <code>-</code>	<code>eval_unary</code>

Operator `+` is overloaded:

- `INTEGER + INTEGER -> arithmetic`.
- `STRING + STRING -> concatenation`.

- `STRING + INTEGER` -> concatenation (integer is converted).
- `PATTERN + PATTERN` -> pattern concatenation.

Operator `&` is the pattern conjunction (no-backtrack); when written between pattern terms it sets `no_backtrack` on the right operand.

3.5 Variable scope

Two layers:

- **Local scopes** - one frame per procedure call, lookup walks innermost-first (`tpu.cpp:2085-2092`).
- **Global** - `parser.variables` (`tpu.cpp:2077`).

Assignment to an existing local rebinds in that frame; assignment to a name not in any active frame creates/updates a global (see `assign_var`, `tpu.cpp:2097`).

`LOCAL name [:= expr], ...` forces creation in the innermost frame even when an outer name exists (`declare_local`, `tpu.cpp:2107`).

3.6 Compile / interpret pipeline

For a complete tour see section 12.

- `compile_text` (`tpu.cpp:2512`) splits the source into statements, recognises top-level `PROCEDURE ... ENDPROCEDURE` blocks, and runs everything else immediately.
- `parse_block` (`tpu.cpp:2291`) recognises `IF`, `LOOP`, `EXITIF`, `RETURN`, `ABORT`, `BREAK`, `ON_ERROR`, `LOCAL`, and any other statement (as a `CALL`).
- `execute_block` (`tpu.cpp:6030`) walks the Stmt tree, dispatching each Stmt by Kind and honouring the FlowSignal state machine.
- `execute_statement` (`tpu.cpp:5212`) is the runtime workhorse: recognises the assignment form, extracts the call name and parenthesised argument text, takes an undo snapshot, and dispatches by name to one of about 100 built-in branches.

4. Built-In Procedures (the API)

This section lists every built-in procedure the dispatcher recognises in `tpu.cpp`. There are two dispatchers:

- **Statement (effectful) built-ins** - matched in `execute_statement` from `tpu.cpp:5331` onwards. These are invoked as commands and may not return a value.
- **Expression built-ins** - matched in `eval_function` from `tpu.cpp:6468`. These produce a `Value`.

Many names appear in both dispatchers (e.g. `MARK`, `BEGINNING_OF`, `SEARCH`, `GET_INFO`). When called as an expression they return a `Value`; when called as a statement they side-effect.

The legend in the **Status** column:

- **F** - fully implemented (in the sense of: behaves like DEC TPU for the documented arguments).
- **P** - partial (subset of DEC TPU semantics or one argument shape).
- **S** - stub: name is recognised but the body is empty or a no-op.

4.1 Cursor / motion

Built-in	Synopsis	Status	Notes (line)
<code>MOVE_VERTICAL(n)</code>	Move cursor <code>n</code> lines (negative = up)	F	<code>tpu.cpp:5331</code>
<code>MOVE_HORIZONTAL(n)</code>	Move cursor <code>n</code> columns (negative = left)	F	<code>tpu.cpp:5332</code>
<code>CURSOR_VERTICAL(n)</code>	Synonym of <code>MOVE_VERTICAL</code>	F	<code>tpu.cpp:5333</code>
<code>CURSOR_HORIZONTAL(n)</code>	Synonym of <code>MOVE_HORIZONTAL</code>	F	<code>tpu.cpp:5334</code>
<code>SCROLL([win\ buf,] n)</code>	Scroll the viewport <code>n</code> lines (<code>top_line</code>)	F	Accepts window, buffer, or amount-only
<code>SHIFT([win\ buf,] n)</code>	Shift text/view <code>n</code> columns horizontally	F	Window/buffer + amount forms
<code>POSITION(target)</code>	Move cursor / switch buffer; see below	F	<code>tpu.cpp:5422</code>
<code>BEGINNING_OF([buf])</code>	Go to top of buffer (or return marker)	F	<code>tpu.cpp:5454</code> , <code>tpu.cpp:6699</code>
<code>END_OF([buf])</code>	Go to end of buffer (or return marker)	F	<code>tpu.cpp:5455</code> , <code>tpu.cpp:6708</code>
<code>TOP</code>	Synonym of <code>BEGINNING_OF(CURRENT_BUFFER)</code>	F	<code>tpu.cpp:5454</code>
<code>BOTTOM</code>	Synonym of <code>END_OF(CURRENT_BUFFER)</code>	F	<code>tpu.cpp:5455</code>
<code>LINE_BEGIN</code>	Move to column 0 (or pattern operator)	F	<code>tpu.cpp:5456</code>
<code>LINE_END</code>	Move to end of current line	F	<code>tpu.cpp:5457</code>

POSITION accepts:

- a BUFFER -> make it the current buffer (tpu.cpp:5432)
- a MARKER -> jump to that marker's buffer/row/column
- a RANGE -> jump to the start of the range
- a KEYWORD (TOP , BOTTOM , LINE_BEGIN , LINE_END , BEGINNING_OF , END_OF)

4.2 Text editing

Built-in	Synopsis	Status	Source
SPLIT_LINE	Split current line at cursor	F	tpu.cpp:5349
APPEND_LINE	Append the next line to this one (delete newline)	F	tpu.cpp:5350
ERASE_LINE	Delete the current line	F	tpu.cpp:5351
ERASE	Delete current line, range, or buffer contents	F	tpu.cpp:5459
ERASE_CHARACTER(n)	Delete n characters at/before cursor (signed)	F	tpu.cpp:5352
COPY_TEXT(arg)	Insert text at cursor (string, range, or selection)	F	tpu.cpp:5384
MOVE_TEXT([range])	Cut to paste buffer; uses SELECT_RANGE if no arg	F	tpu.cpp:5383
FILL	Fill SELECT_RANGE within margins	F	tpu.cpp:5353
CHANGE_CASE(arg, k)	UPPER / LOWER / INVERT on arg (range or buffer)	F	tpu.cpp:5359
TRANSLATE(range\ buf, s, d)	Per-character substitution of s -> d over a range or buffer	F	Range and buffer forms
READ_FILE(path)	Insert file contents at cursor	F	
WRITE_FILE(...)	See notes below	F	DEC + host forms
UNDO	Pop one snapshot from the undo stack	F	tpu.cpp:5700
REDO	Pop one snapshot from the redo stack	F	

WRITE_FILE forms:

- **DECTPU:** WRITE_FILE(buffer [, filespec]) and WRITE_FILE(range [, filespec])
- **Host / EVE convenience:** WRITE_FILE("path") writes the current buffer (bare string path alone is **invalid** on OpenVMS DECTPU).

4.3 Mark / selection / range

Built-in	Synopsis	Status	Source
<code>SELECT</code>	Drop a mark at the cursor (start the active selection)	F	
<code>SELECT_RANGE</code>	Realise the active selection as a <code>RANGE</code> value	F	
<code>MARK ([FREE_CURSOR])</code>	Statement: drop a mark; expression: return cursor Marker. <code>FREE_CURSOR</code> enables virtual columns past EOL	F	Also <code>SET (FREE_CURSOR, ON OFF)</code>
<code>CREATE_RANGE (a, b)</code>	Build a <code>RANGE</code> from two markers (or marker pair)	F	
<code>MODIFY_RANGE (r, start, end)</code>	Adjust range endpoints (markers or <code>LINE_ / BUFFER_</code> keywords)	F	

4.4 Search / match

Built-in	Synopsis	Status	Source
<code>SEARCH (pat [, dir])</code>	Statement form: search and report; expression form: <code>RANGE</code>	F	<code>tpu.cpp:5392</code> , <code>tpu.cpp:6727</code>
<code>SEARCH_QUIETLY (pat [, dir])</code>	Same but no status message	F	same
<code>MATCH (pat)</code>	Test pattern at cursor; sets message	F	<code>tpu.cpp:5413</code>

`pat` may be a `STRING` (treated as a literal substring) or a `PATTERN` value built up with the pattern operators. The optional `dir` is `FORWARD` (default - actually whatever the buffer's `reverse_direction` flag says) or `REVERSE`. See `parse_pattern_expression` (`tpu.cpp:5095-5182`).

4.5 Buffer / file management

Built-in	Synopsis	Status	Source
<code>CREATE_BUFFER (name [, file])</code>	Create a new buffer; optional file path	F	<code>tpu.cpp:6748</code>
<code>GET_BUFFER (name)</code>	Look up a buffer by name (synonym <code>BUFFER</code>)	F	<code>tpu.cpp:6741</code>
<code>BUFFER (name)</code>	Same	F	<code>tpu.cpp:6741</code>
<code>DELETE (buf)</code>	Delete the buffer	F	<code>tpu.cpp:5822</code>
<code>DELETE (range)</code>	Erase the range	F	<code>tpu.cpp:5822</code>

Built-in	Synopsis	Status	Source
<code>DELETE(integer)</code>	Erase n characters at cursor	F	<code>tpu.cpp:5847</code>
<code>POSITION(buffer)</code>	Make the buffer current	F	<code>tpu.cpp:5432</code>

4.6 Windows

Built-in	Synopsis	Status	Source
<code>CREATE_WINDOW(top, height [, status_line])</code>	Make a new window viewport	F	<code>tpu.cpp:5858</code>
<code>MAP(window, buffer)</code>	Bind a buffer to a window	F	<code>tpu.cpp:5866</code>
<code>UNMAP(window)</code>	Detach buffer from window (renders blank)	F	<code>tpu.cpp:5875</code>
<code>ADJUST_WINDOW(win, dtop, dbot)</code>	Resize / reposition by delta	F	<code>tpu.cpp:5879</code>
<code>SPLIT_WINDOW</code>	Split current window horizontally	F	<code>tpu.cpp:5886</code>
<code>SPLIT_WINDOW_VERTICAL</code>	Split side-by-side	F	<code>tpu.cpp:5887</code>
<code>NEXT_WINDOW</code>	Rotate focus through windows	F	<code>tpu.cpp:5888</code>
<code>DELETE_WINDOW</code>	Remove the current window	F	<code>tpu.cpp:5889</code>

4.7 Keymap and key binding

Built-in	Synopsis	Status	Source
<code>CREATE_KEY_MAP(name)</code>	Allocate a fresh empty key map	F	command verb at <code>tpu.cpp:3565</code>
<code>CREATE_KEY_MAP_LIST(name)</code>	Allocate a new key-map list	F	<code>tpu.cpp:3571</code>
<code>ADD_KEY_MAP(list, FIRST LAST, map)</code>	Push a map onto a list	F	<code>tpu.cpp:3578</code>
<code>REMOVE_KEY_MAP(list, map)</code>	Remove a map from a list	F	<code>tpu.cpp:3580</code>
<code>DEFINE_KEY(key, tpu-text)</code>	Bind a key to a TPU statement	F	<code>tpu.cpp:3563</code>
<code>UNDEFINE_KEY(key)</code>	Remove a binding	F	<code>tpu.cpp:3582</code>
<code>LOOKUP_KEY(key)</code>	Show the current binding for a key	F	<code>tpu.cpp:3584</code>
<code>KEY_NAME(key)</code>	Return / echo the normalised name	P	<code>tpu.cpp:3586</code>

Built-in	Synopsis	Status	Source
<code>LAST_KEY</code>	Most recent keycode	F	<code>tpu.cpp:3589</code>
<code>READ_KEY</code>	Wait for one keystroke; bind to <code>LAST_KEY</code>	F	<code>tpu.cpp:3591</code>

The DCL-level commands `CREATE_KEY_MAP` etc. are dispatched at the command-line prompt (`Ctrl-Z`); when invoked as expressions or statements from a TPU script they go through `execute_command`.

4.8 SET (attribute) - the universal setter

`SET (attribute [, value ...])` configures a flag or value on the current buffer or globally. See full reference in section 5. Source: `tpu.cpp:5529-5687`.

4.9 Status / introspection

These can be called as commands (set the message line) or as expressions (return a Value).

Built-in	Returns	Source
<code>CURRENT_LINE</code>	The current line as a string	<code>tpu.cpp:6693</code>
<code>CURRENT_COLUMN</code>	1-based column number	<code>tpu.cpp:6694</code>
<code>CURRENT_ROW</code>	1-based row number	<code>tpu.cpp:6695</code>
<code>CURRENT_OFFSET</code>	1-based offset within the current line	<code>tpu.cpp:6696</code>
<code>CURRENT_BUFFER</code>	The active <code>Buffer</code>	<code>tpu.cpp:6697</code>
<code>CURRENT_CHARACTER</code>	One-character string of the byte under the cursor	<code>tpu.cpp:6865</code>
<code>CURRENT_DIRECTION</code>	<code>FORWARD</code> or <code>REVERSE</code> keyword	<code>tpu.cpp:6869</code>
<code>CURRENT_WINDOW</code>	First-class <code>WINDOW</code> value for the focused viewport	
<code>LAST_KEY</code>	The most recent integer keycode	
<code>ERROR</code>	Keyword of the last recorded status (<code>TPU\$...</code>)	
<code>ERROR_LINE</code>	1-based row of the last error	
<code>ERROR_TEXT</code>	Text of the last status message	
<code>DEBUG_LINE</code>	(statement only) prints the row of the cursor	

4.10 GET_INFO

`GET_INFO(subject, "what")` queries an attribute. The subject may be a `BUFFER`, `RANGE`, `MARKER`, `WINDOW`, the keyword `SYSTEM`, or an integer (`0` for Sector7 system/extension queries). The selector is a string.

Buffer selectors (representative):

RECORD_COUNT / LENGTH / LINES	line count
MODIFIED	0 or 1
MODIFIABLE	0 or 1
NO_WRITE	0 or 1
PERMANENT	0 or 1
SYSTEM	0 or 1
JOURNALING	0 or 1
JOURNAL_FILE	journal path
NAME	buffer name
FILE_NAME / INPUT_FILE / OUTPUT_FILE	backing path
LEFT_MARGIN	integer
RIGHT_MARGIN	integer
MAX_LINES	integer (0 = unlimited)
EOB_TEXT	string
DIRECTION	FORWARD/REVERSE
INSERT_MODE	INSERT/OVERSTRIKE
KEY_MAP_LIST	string
WRAP	0 or 1
CURRENT_COLUMN / COLUMN	1-based
CURRENT_ROW / ROW	1-based
OFFSET	1-based
VISIBLE_TOP / VISIBLE_BOTTOM	visible window rows
TOP / ORIGINAL_TOP	1
BOTTOM / ORIGINAL_BOTTOM	last line number
BOUND / BEFORE_BOL / BEYOND_EOL / MIDDLE_OF_TAB	App.C probes

DECTPU system subject (`GET_INFO(SYSTEM, ...)`):

version	integer (Sector7 reports TPU_SYSTEM_VERSION,
currently 1)	
update	integer (Sector7 reports TPU_SYSTEM_UPDATE,
currently 0)	

On OpenVMS DECTPU these return **3** and **2** respectively (verified on `t86`).

Sector7 extension (`GET_INFO(0, ...)` — not valid DECTPU):

VERSION	release string, e.g. "V1.08-0"
PLATFORM	"WINDOWS"/"MACOS"/"LINUX"/"POSIX"
RELEASE_DATE	"YYYY-MM-DD"
SCREEN_HEIGHT / SCREEN_ROWS	integer
SCREEN_WIDTH / SCREEN_COLS	integer
WINDOW_COUNT / BUFFER_COUNT / PROCEDURE_COUNT	
MOUSE_ROW / MOUSE_COLUMN	1-based

Range subject:

BEGIN / START	Marker (start of range)
END	Marker (end of range)
BUFFER	Buffer
LENGTH / RECORD_COUNT	line span

Marker subject:

ROW	1-based row
COLUMN	1-based column
BUFFER	buffer
FREE_CURSOR	0 or 1

Window subject:

BUFFER	mapped buffer
VISIBLE / STATUS_LINE	flags
VISIBLE_TOP / height / width	geometry

4.11 Pattern construction

Pattern primitives produce real `Value::PATTERN` values; `+` and `&` on patterns concatenate. See `tpu.cpp:6970-6995`.

Built-in	Returns
<code>ANCHOR</code>	Empty pattern with <code>anchored=true</code>
<code>UNANCHOR</code>	Empty pattern with <code>anchored=false</code>
<code>LINE_BEGIN</code>	Single token: match start of line
<code>LINE_END</code>	Single token: match end of line
<code>PAGE_BREAK</code>	Match a form-feed character (<code>\f</code>)
<code>REMAIN</code>	Match the rest of the current line
<code>ANY("set")</code>	One character from the set
<code>NOTANY("set")</code>	One character not in the set
<code>ARB(n)</code>	Exactly <code>n</code> arbitrary characters
<code>SCAN("set")</code>	Skip until next character in the set (at least one)
<code>SCANL("set")</code>	SCAN, but empty match allowed

Built-in	Returns
<code>SPAN("set")</code>	Maximal span of characters in the set
<code>SPANL("set")</code>	SPAN, but empty match allowed

4.12 Messages and miscellaneous

Built-in	Behaviour	Status
<code>MESSAGE(s)</code>	Print <code>s</code> on the message line and append to <code>\$MESSAGE_BUFFER</code> . If <code>s</code> is a <code>TPU\$...</code> code (or bare App. B abbrev), expand facility text first.	F
<code>MESSAGE_TEXT(code)</code>	Return (expression) or display (statement) Appendix B text for <code>TPU\$...</code> / bare abbrev	F
<code>SHOW(topic)</code>	Run a SHOW command (<code>BUFFERS</code> , <code>KEYMAPS</code> , <code>PROCEDURES</code> , <code>VARIABLES</code> , <code>DEFAULTS</code> , <code>VERSION</code> , <code>PASTES</code> , ...)	F
<code>BELL</code>	Ring the terminal bell	F
<code>REFRESH / UPDATE</code>	Force a screen redraw	F
<code>SLEEP(n)</code>	Sleep <code>n</code> seconds	F
<code>LOCATE_MOUSE</code>	Display last reported mouse coordinates	F
<code>STORE_PASTE(name)</code>	Save the current unnamed paste slot under <code>name</code>	F
<code>RECALL_PASTE(name)</code>	Load named paste into the slot; expression form returns the text	F

Appendix B catalog: ~325 symbols in `tpu_messages_appendix_b.inc` (regenerate with `docs/extract_appendix_b_messages.py`). Human-readable table: `docs/TPU_MESSAGES_APPENDIX_B.md`.

4.13 Procedure / control / exit

Built-in	Behaviour	Source
<code>EXECUTE(target)</code>	If <code>target</code> is a <code>PROGRAM</code> , run its body; if <code>LEARN</code> , replay its statements; otherwise treat as a procedure name to dispatch	<code>tpu.cpp:5503</code>
<code>RETURN</code>	Raise the <code>RETURN_</code> flow signal	<code>tpu.cpp:5518</code>
<code>ABORT</code>	Raise the <code>ABORT_</code> flow signal (handled by enclosing <code>ON_ERROR</code>)	<code>tpu.cpp:5519</code>
<code>BREAK</code>	Raise the <code>BREAK_</code> flow signal (exits one LOOP level)	<code>tpu.cpp:5520</code>
<code>EXIT [n]</code>	Quit the editor; optional integer becomes the process exit code	<code>tpu.cpp:5521</code>
<code>QUIT [n]</code>	Synonym	<code>tpu.cpp:5521</code>

4.14 String / numeric utility (expression-only)

Built-in	Returns	Source
<code>STR(v)</code>	Stringification (via <code>as_string</code>)	<code>tpu.cpp:6468</code>
<code>INT(v)</code>	Integer coercion	<code>tpu.cpp:6469</code>
<code>ASCII(s)</code>	Integer ASCII code of the first character of <code>s</code>	<code>tpu.cpp:6470</code>
<code>LENGTH(s)</code>	Length of <code>s</code> (string) in characters	<code>tpu.cpp:6475</code>
<code>INDEX(haystack, needle)</code>	1-based offset of <code>needle</code> in <code>haystack</code> , or 0 if not found	<code>tpu.cpp:6476</code>
<code>SUBSTR(s, offset, len)</code>	Substring starting at 1-based <code>offset</code> of length <code>len</code>	<code>tpu.cpp:6481</code>
<code>EDIT(s, UPPER LOWER)</code>	Case-fold a string	<code>tpu.cpp:6490</code>
<code>FAO(format, ...)</code>	DEC's "Formatted ASCII Output" - integer/string formatting; see section 5 for directives	<code>tpu.cpp:6500</code>
<code>HELP_TEXT(topic)</code>	Returns the help text for a topic as a string	<code>tpu.cpp:6803</code>

4.15 Reading user input

Built-in	Behaviour	Source
<code>READ_LINE([prompt])</code>	Prompt for a line of text; returns string	
<code>READ_CHAR([prompt])</code>	Read a single character; returns integer ASCII	
<code>PROMPT([prompt] [, default] [, NOECHO ECHO])</code>	Character-cell PROMPT: optional default (used when empty / headless) and echo control	

4.16 Test harness

Built-in	Behaviour	Source
<code>ASSERT(expr [, label])</code>	Pass when <code>expr</code> is truthy	<code>tpu.cpp:5725</code>
<code>ASSERT_TRUE(expr [, label])</code>	Synonym	same
<code>ASSERT_FALSE(expr [, label])</code>	Pass when <code>expr</code> is falsy	<code>tpu.cpp:5739</code>
<code>ASSERT_EQ(actual, expected [, label])</code>	Pass when both are equal (<code>==</code> for INTEGER, <code>as_string</code> otherwise)	<code>tpu.cpp:5753</code>
<code>TEST "label"</code>	Start a new test group; prints a header in <code>--test</code> mode	<code>tpu.cpp:5773</code>

4.17 LEARN

Built-in	Behaviour	Source
<code>LEARN_BEGIN</code>	Start recording subsequent statements into a learn buffer	
<code>LEARN_END</code>	Stop recording; returns the LEARN value (sequence of statements)	
<code>LEARN_ABORT</code>	Discard the in-progress recording	
<code>EXPAND_LEARN(learn)</code>	Convert a LEARN value into a PROGRAM value	

A LEARN value is replayed with `EXECUTE(seq)`.

4.18 Journal

Built-in	Behaviour	Source
<code>JOURNAL_OPEN(path)</code>	Truncate <code>path</code> and start writing each subsequent statement	<code>tpu.cpp:5902</code>
<code>JOURNAL_CLOSE</code>	Stop journaling	<code>tpu.cpp:5912</code>
<code>RECOVER_BUFFER([path])</code>	Replay the journal at <code>path</code> (or the buffer's current <code>journal_path</code>)	<code>tpu.cpp:5917</code>

The journal is a plain text TPU script (one statement per line), not a binary form. See section 9.

4.19 External processes

Built-in	Behaviour	Source
<code>CREATE_PROCESS(cmd [, outbuf])</code>	Fork+exec <code>cmd</code> via <code>/bin/sh -c</code> ; route stdout/stderr to <code>outbuf</code> (default: current)	
<code>SEND(text [, process])</code>	Send <code>text + "\n"</code> to the process's stdin	
<code>SEND_EOF([process])</code>	Close the process's stdin	
<code>ATTACH([process])</code>	Forward subsequent keyboard input to the process until Ctrl-Z detach or Ctrl-D EOF	
<code>SPAWN(cmd)</code>	Run host shell command via <code>system()</code> (Linux); interactive form suspends ncurses	

While `ATTACH`ed, printable keys and Enter go to the child stdin. Process exit clears the attach slot automatically.

4.20 Arrays

Built-in	Behaviour	Source
<code>CREATE_ARRAY</code>	<code>CREATE_ARRAY</code> or <code>CREATE_ARRAY(n [, start])</code> ; typed <code>{key}</code> slots	F

Built-in	Behaviour	Source
Array walk / size	<code>GET_INFO(arr, "FIRST" \ "NEXT" \ ... \ "SIZE") ;</code> <code>GET_INFO (ARRAY, ...)</code> global list	F
Clear / delete	<code>arr{k}:=UNSPECIFIED; DELETE(arr)</code> (refs → UNSPECIFIED)	F

Subscript access is by `a{"key"}`. Iteration / size / clear primitives are **not** implemented; see section 13.

4.21 COMPILE / PROGRAM

Built-in	Behaviour	Source
<code>COMPILE(source-string)</code>	Compile a TPU source string and return a PROGRAM Value	<code>tpu.cpp:6766</code>
<code>EXECUTE(program)</code>	Run a previously-compiled PROGRAM (or LEARN sequence, or procedure name)	<code>tpu.cpp:6775</code>

4.22 Clipboard

Built-in	Behaviour	Source
<code>WRITE_CLIPBOARD([text])</code>	Send to the host clipboard (pbcopy/xclip/wl-copy)	<code>tpu.cpp:5973</code>
<code>READ_CLIPBOARD /</code> <code>GET_CLIPBOARD</code>	Read the host clipboard into the current buffer's paste-text slot	<code>tpu.cpp:5983,</code> <code>tpu.cpp:6809</code>

4.23 File-name helpers

Built-in	Behaviour	Status
<code>FILE_PARSE(spec)</code>	Unix path normalisation / components	F
<code>FILE_SEARCH(spec)</code>	Search for a file; returns path or empty	F
<code>EXPAND_NAME(spec)</code>	Expand <code>~</code> and relative paths	F

4.24 DECwindows shim (accepted-no-op)

The following names are accepted as statements but do nothing in the ncurses build (they exist only so DECwindows-flavoured scripts compile): `DEFINE_WIDGET_CLASS`, `CREATE_WIDGET`, `MANAGE_WIDGET`, `RAISE_WIDGET`, `LOWER_WIDGET`, `REALIZE_WIDGET`, `UNMANAGE_WIDGET`, `READ_GLOBAL_SELECT`, `WRITE_GLOBAL_SELECT`, `SEND_CLIENT_MESSAGE`, `GET_DEFAULT`, `GET_GLOBAL_SELECT`, `CONVERT`.
Status: S.

4.25 CALL_USER

`CALL_USER(routine, parameter [, library])` loads a shared object (POSIX `dlopen` / `dlsym`) and calls a C-linkage `const char* (*) (const char*)` symbol, returning the result string.

5. Built-in Constants, Keywords, and System Variables

5.1 Direction / mode keywords

These are returned as `KEYWORD` values when the bare name is looked up and the variable lookup fails. The receiving built-in inspects the keyword.

Keyword	Used by
<code>FORWARD</code>	<code>SEARCH</code> , <code>SET (FORWARD)</code> , default direction
<code>REVERSE</code>	<code>SEARCH</code> , <code>SET (REVERSE)</code>
<code>ON</code> , <code>TRUE</code>	Truthy <code>SET</code> arguments
<code>OFF</code> , <code>FALSE</code>	Falsy <code>SET</code> arguments
<code>UPPER</code> , <code>LOWER</code> , <code>INVERT</code>	<code>CHANGE_CASE</code> , <code>EDIT</code>
<code>INSERT</code> , <code>OVERSTRIKE</code>	<code>SET (INSERT)</code> / <code>SET (OVERSTRIKE)</code>

5.2 Pattern keywords

See section 4.11.

<code>ANCHOR</code>	<code>UNANCHOR</code>	<code>LINE_BEGIN</code>	<code>LINE_END</code>	<code>PAGE_BREAK</code>	<code>REMAIN</code>	
<code>ANY</code>	<code>NOTANY</code>	<code>ARB</code>	<code>SCAN</code>	<code>SCANL</code>	<code>SPAN</code>	<code>SPANL</code>

5.3 Position keywords

<code>TOP</code>	<code>BOTTOM</code>	<code>BEGINNING_OF</code>	<code>END_OF</code>	<code>LINE_BEGIN</code>	<code>LINE_END</code>	<code>EOL</code>	<code>BOL</code>
------------------	---------------------	---------------------------	---------------------	-------------------------	-----------------------	------------------	------------------

`EOL` and `BOL` are accepted as synonyms of `LINE_END` and `LINE_BEGIN` inside the pattern parser; outside it they are unresolved and produce `KEYWORD` values.

5.4 System buffers

Special buffers automatically created by the editor:

Name	Created by	Contents
<code>MAIN</code>	<code>main()</code> at start	The file passed on the command line (or empty)
<code>\$MESSAGE_BUFFER</code>	First <code>MESSAGE</code>	Append-only log of every message (capped at 1000 lines)
<code>\$SHOW_BUFFERS</code>	<code>SHOW BUFFERS</code>	The buffer-list output

Name	Created by	Contents
<code>\$SHOW_KEYMAPS</code>	<code>SHOW KEYMAPS</code>	Active key map list
<code>\$SHOW_PROCEDURES</code>	<code>SHOW PROCEDURES</code>	All compiled user procedures
<code>\$SHOW_VARIABLES</code>	<code>SHOW VARIABLES</code>	All global variable bindings
<code>\$SHOW_DEFAULTS</code>	<code>SHOW DEFAULTS</code>	Per-buffer attributes
<code>\$SHOW_SUMMARY</code>	<code>SHOW SUMMARY</code>	Summary table
<code>\$SHOW_SYSTEM</code>	<code>SHOW SYSTEM</code>	Screen / mouse / timer state
<code>HELP</code>	<code>Ctrl-Z HELP</code>	Rendered help text

5.5 System variables

Implicit names looked up by the dispatcher:

Name	Returns
<code>CURRENT_BUFFER</code>	The active buffer
<code>CURRENT_WINDOW</code>	The active window (<code>WINDOW</code> value)
<code>CURRENT_LINE</code>	Text of the line under the cursor
<code>CURRENT_COLUMN</code>	1-based column number
<code>CURRENT_ROW</code>	1-based row number
<code>CURRENT_OFFSET</code>	1-based offset
<code>CURRENT_DIRECTION</code>	<code>FORWARD</code> or <code>REVERSE</code> keyword
<code>CURRENT_CHARACTER</code>	Character under cursor
<code>LAST_KEY</code>	Most recent keycode
<code>MARK ([FREE_CURSOR])</code>	Marker at cursor; optional free-cursor mode

`MARK (FREE_CURSOR)` and `SET (FREE_CURSOR, ON|OFF)` allow the cursor to sit past end-of-line (App.C-style padding on insert).

5.6 SET attributes - complete reference

`SET (key [, value ...])` configures an attribute. The dispatcher is at `tpu.cpp:5529-5687`. Attributes are split into "wired" (the editor actually does something) and "accepted no-op" (so scripts compile).

5.6.1 Wired (active) attributes

Attribute	Argument shape	Effect
INSERT	none	Set INSERT mode
OVERSTRIKE	none	Set OVERSTRIKE mode
WRAP	[buffer,] ON OFF	Auto-wrap on type
NO_WRAP	none	Auto-wrap off
BACKUP	[ON OFF]	Whether WRITE_FILE renames existing target to .bak
FORWARD	none	Default search direction = forward
REVERSE	none	Default search direction = reverse
MODIFIED	[buffer,] ON OFF	Force-set the modified flag
MODIFIABLE	[buffer,] ON OFF	Permit edits
NO_WRITE	[buffer,] ON OFF	Suppress WRITE_FILE
PERMANENT	none	Buffer cannot be deleted
SYSTEM	none	Buffer marked as system-internal
JOURNALING	[buffer,] ON OFF	Per-buffer journal toggle
EOB_TEXT	[buffer,] "text"	Text shown one row past last line
LEFT_MARGIN	[buffer,] n	1-based left margin
RIGHT_MARGIN	[buffer,] n	1-based right (wrap) margin
MARGINS	[buffer,] l, r	Both at once
MAX_LINES	[buffer,] n	Cap the buffer at n lines (0 = unlimited)
TAB_STOPS	[buffer,] n1, n2, ...	Custom tab columns; empty = every 8
LINE_NUMBER	[ON OFF]	Toggle the gutter
BELL	[ON OFF]	Editor bell on/off
KEY_MAP_LIST	"list-name" [, buffer]	Select key map list (per buffer)
MOUSE	[ON OFF]	Enable ncurses mouse events
TIMER	ms, "TPU statement"	Fire the TPU statement every ms ms
PRE_KEY_PROCEDURE	"text"	Run before every key dispatch
POST_KEY_PROCEDURE	"text"	Run after every key dispatch

Attribute	Argument shape	Effect
UNDEFINED_KEY	"text"	Run when a key has no binding
SHIFT_KEY	KEY	Secondary GOLD-style prefix
VIDEO	range, attr-kw ... or OFF	Paint a region with BOLD/REVERSE/UNDERLINE/...
STATUS_LINE	window, ON OFF TEXT [, format]	Status line on per window
FREE_CURSOR	[ON OFF]	Allow virtual columns past EOL (App.C padding)

5.6.2 Accepted-no-op attributes

These keys are syntactically accepted (no error) but produce no effect in the ncurses build:

```
WIDTH, HEIGHT, PAD, TEXT, SCROLLING, SCREEN_UPDATE, PROMPT_AREA,
DISPLAY_VALUE, TRACEBACK, DEBUG, INFORMATIONAL, SUCCESS,
MESSAGE_FLAGS, AUTO_REPEAT, FACILITY_NAME, DEFAULT_DIRECTORY,
RECORD_MODE, RECORD_ATTRIBUTE, LEFT_MARGIN_ACTION,
RIGHT_MARGIN_ACTION, COLUMN_MOVE_VERTICAL, CROSS_WINDOW_BOUNDS,
DETACHED_ACTION, MOVE_VERTICAL_CONTEXT, MESSAGE_ACTION_LEVEL,
MESSAGE_ACTION_TYPE, SELF_INSERT, SPECIAL_ERROR_SYMBOL,
ERASE_UNMODIFIABLE, OUTPUT_FILE, KEYSTROKE_RECOVERY
```

(See `tpu.cpp:5667-5684`. Unknown attributes outside this list produce the message `SET: unknown attribute <key>` and a non-fatal warning.)

5.7 FAO directives

The `FAO(format, ...)` built-in (Formatted ASCII Output - DEC's analogue of `printf`) is implemented at `tpu.cpp:6500-6692`. Directives start with `!`.

Two-letter directives (consume one argument each):

Directive	Effect
!AS	String, right-padded with spaces
!AC	String (synonym of !AS)
!AD	String, truncated to width
!ZL	Integer, zero-padded
!SL	Signed integer (always includes sign)

Directive	Effect
<code>!SW</code>	Synonym of <code>!SL</code>
<code>!UL</code>	Unsigned integer
<code>!UW</code>	Unsigned word
<code>!UB</code>	Unsigned byte
<code>!XL</code>	Hex (upper case)
<code>!XW</code>	Hex word
<code>!XB</code>	Hex byte
<code>!OL</code>	Octal
<code>!OW</code>	Octal word
<code>!OB</code>	Octal byte
<code>!BL</code>	Binary
<code>!BW</code>	Binary word
<code>!BB</code>	Binary byte
<code>!%S</code>	Plural <code>s</code> based on the previous argument
<code>!%D</code>	Date <code>dd-Mon-YYYY</code>
<code>!%T</code>	Time <code>HH:MM:SS</code>
<code>!%C</code>	One ASCII character (the integer modulo 0x80)

Single-character directives:

Directive	Effect
<code>!!</code>	Literal <code>!</code>
<code>! /</code>	Newline
<code>! _</code>	Tab
<code>! ^</code>	Form-feed
<code>!@</code>	Right-justify next argument in <code>count</code> columns
<code>!*x</code>	Fill <code>count</code> copies of literal <code>x</code>
<code>!+</code>	Skip the next argument (or <code>count</code> args)
<code>!-</code>	Rewind the argument pointer

Directive	Effect
<code>!nFXX</code>	Repeat directive <code>XX</code> <code>n</code> times

A leading integer between `!` and the directive sets the width (for the `Z/X/O/B/S/U/A` directives) or repetition count (for `F`, `@`, `*`). The width may be supplied as an argument by writing `!#XX`, which consumes the next argument as the width.

Examples (from `tests/tpu_tests.tpu`):

```
FAO("hi !AS", "joe")      -> "hi joe"
FAO("n=!ZL", 42)          -> "n=42"
FAO("n=!4ZL", 42)         -> "n=0042"
FAO("hex=!XL", 255)       -> "hex=FF"
FAO("hex2=!2XL", 5)       -> "hex2=05"
FAO("neg=!SL", -3)        -> "neg=-3"
FAO("plus=!SL", 7)        -> "plus=+7"
FAO("!8@", "hi")          -> "      hi"    ! right-justify w8
FAO("!2*x")               -> "xx"         ! repeat fill
FAO("!+!ZL", 10, 20)      -> "20"         ! skip then format
FAO("!2FZL", 1, 2, 3)     -> "12"         ! repeat !ZL twice
```

6. Buffer / Window / Range / Marker Model

6.1 Buffers

A `Buffer` (see `tpu.cpp:531`) is a vector of lines plus cursor and flags. Lines do not carry trailing newlines.

6.1.1 Creation

```
buf := CREATE_BUFFER("WORK");
buf := CREATE_BUFFER("logfile", "/tmp/log");
```

The two-argument form pre-loads from the file if it exists; otherwise the buffer is empty and the path is remembered for the next `WRITE_FILE`.

6.1.2 Lifecycle

```
DELETE(buf);
```

Deletes the buffer (unless `permanent`). If `buf` was current, the first remaining buffer becomes current. Any window mapped to it is re-mapped to the new current buffer.

6.1.3 Attributes

All attributes are settable via `SET (key [, value], buffer)`.

Attribute	Default	Field in C++
<code>modified</code>	false	<code>Buffer::modified</code>
<code>insert_mode</code>	true (INSERT)	<code>Buffer::insert_mode</code>
<code>writable</code>	true	<code>Buffer::writable</code>
<code>no_write</code>	false	<code>Buffer::no_write</code>
<code>permanent</code>	false	<code>Buffer::permanent</code>
<code>system_buffer</code>	false	<code>Buffer::system_buffer</code>
<code>reverse_direction</code>	false (FORWARD)	<code>Buffer::reverse_direction</code>
<code>journaling</code>	false	<code>Buffer::journaling</code>
<code>left_margin</code>	1	<code>Buffer::left_margin</code>
<code>right_margin</code>	80	<code>Buffer::right_margin</code>
<code>max_lines</code>	0 (unlimited)	<code>Buffer::max_lines</code>

Attribute	Default	Field in C++
<code>tab_stops</code>	empty (every 8)	<code>Buffer::tab_stops</code>
<code>wrap_on_type</code>	false	<code>Buffer::wrap_on_type</code>
<code>make_backup</code>	true	<code>Buffer::make_backup</code>
<code>eob_text</code>	"[End of file]"	<code>Buffer::eob_text</code>
<code>keymap_list_name</code>	empty	<code>Buffer::keymap_list_name</code>

6.1.4 Coordinate convention

Internally, rows and columns are **0-based**. All user-visible positions (`CURRENT_ROW`, `CURRENT_COLUMN`, `GO TO <n>`, `GET_INFO` selectors) are **1-based**. Conversion happens at the API boundary.

6.2 Windows

A `TPUWindow` (see `tpu.cpp:1786`) is a `(top_row, left_col, height, width, buffer*, status_line, status_format)` tuple. Windows are allocated by `create_window`, mapped to buffers by `map_window`, and torn down by `delete_window`.

6.2.1 Tiling

The default geometry is one window of `screen_rows - 2` rows mapped to the `MAIN` buffer. `SPLIT_WINDOW` divides the current window horizontally; `SPLIT_WINDOW_VERTICAL` divides it side-by-side. `NEXT_WINDOW` rotates focus through the list.

6.2.2 ADJUST_WINDOW

`ADJUST_WINDOW(window, dtop, dbot)` adds `dtop` to the top row and `dbot` to the bottom row, clamping to screen extent. See `tpu.cpp:2924`.

6.2.3 Status line

Each window may have its own status line (last row of its allocation). Default format is just the buffer name + cursor; a custom format can be set with `SET (STATUS_LINE, window, TEXT, "format")` and follows the same `FAO` directive vocabulary.

6.3 Markers

A `Marker` (see `tpu.cpp:188`) is a `(buffer, row, col, free_cursor)` quadruple. Markers are produced by:

- `MARK` - the cursor's current position.
- `BEGINNING_OF(buffer)` / `END_OF(buffer)`.
- The endpoints of a `RANGE` via `GET_INFO(range, "BEGIN" | "END")`.

`free_cursor` mirrors DECTPU's free-cursor flag (the marker may sit past end-of-line in virtual whitespace); this implementation honours it on round-trip but the renderer clamps display to line length.

6.4 Ranges

A `Range` (see `tpu.cpp:205`) is a half-open `[begin, end)` span over a single buffer. Produced by:

- `SEARCH(pattern)` - if the search succeeds, a range covering the match is returned.
- `CREATE_RANGE(marker1, marker2)` - explicit construction.
- `SELECT_RANGE` after a `SELECT` and a cursor move.

Ranges are erased with `ERASE(range)`, queried with `GET_INFO(range, "BEGIN" | "END" | "BUFFER" | "LENGTH")`, and accept the position keywords.

6.5 Pattern matching engine

Patterns are a sequence of `TPUPatternToken` (see `tpu.cpp:276`). Each token is a kind plus optional text/count. The matcher in `TPUPattern::match` (`tpu.cpp:344`) walks the input in two passes:

1. The **anchored** flag controls whether the pattern must match at the current cursor or may slide forward looking for a match.
2. Each token consumes from the current position; concatenation is ordered (left to right).

Operators:

Operator	Match
literal <code>"string"</code>	Exact substring
<code>ANCHOR</code>	Match must start here
<code>UNANCHOR</code>	Permit forward sliding within the line/buffer
<code>LINE_BEGIN</code>	Match start-of-line
<code>LINE_END</code>	Match end-of-line
<code>PAGE_BREAK</code>	Form-feed
<code>REMAIN</code>	The rest of the current line
<code>ANY("set")</code>	One character in set
<code>NOTANY("set")</code>	One character not in set
<code>ARB(n)</code>	Exactly n arbitrary characters
<code>SCAN("set")</code>	Slide forward until next character in set; must move at least 1
<code>SCANL("set")</code>	Like SCAN but empty match allowed
<code>SPAN("set")</code>	Maximal run of characters in set

Operator	Match
<code>SPANL("set")</code>	Like SPAN but empty match allowed
<code>+</code>	Concatenate (backtracks allowed)
<code>&</code>	Concatenate, marking the right side as <code>no_backtrack</code>
<code> </code>	Alternation (in <code>parse_pattern_expression</code>)

Backtracking is bounded: the matcher records a small stack of restart points; alternation tries each branch in order. See `TPUPattern::match` (`tpu.cpp:344` - `tpu.cpp:528`).

Examples:

```

SEARCH("foo")                ! literal substring
SEARCH(LINE_BEGIN + "!" + REMAIN) ! comment line
SEARCH(ANCHOR + ANY("abc") + ARB(3)) ! anchored 4-char match
SEARCH(LINE_BEGIN + SPAN("0123456789")) ! leading run of digits

```

7. Section Files, Command Files, and Initialisation

7.1 Portable section files (`#VXSECT`)

VX/TPU writes and loads a **portable section file** (default suffix `.TPU$SECTION`) with magic `#VXSECT 1`. This is **not** OpenVMS DECTPU bytecode; it stores:

- procedure source text (`PROCEDURE ... ENDPROCEDURE`),
- variable and constant **names** (values are not restored — DEC semantics),
- key maps / bindings and key-map lists.

Built-in	Behaviour
<code>SAVE (filespec [, "IDENT", id] ...)</code>	Write section from the current session
<code>RESTORE (filespec)</code>	Load section into the session
<code>EXPAND (filespec)</code>	Dump section text into buffer <code>\$EXPAND</code>

`/SECTION=file` and `--section=` at start-up call `RESTORE`. Optional DEC keywords `NO_DEBUG_NAMES` / `NO_PROCEDURE_NAMES` are accepted and ignored (source form has no separate debug table).

EVE `SAVE` / `SAVE FILE` without parentheses still mean write the **buffer** to disk. Zero-arg EVE `RESTORE` remains a PASTE synonym.

7.2 Command and initialisation files

The closest analogue of `/COMMAND=` and `/INITIAL=` is the start-up file the editor sources automatically. The order is:

1. If `$TPU_STARTUP` is set, source that path.
2. Otherwise, if `$HOME/.tpurc` (or `$USERPROFILE/.tpurc` on Windows) exists, source it.

Source: `tpu.cpp:7257-7276`.

Both files are read as ordinary TPU source - they may declare procedures, install custom key bindings, set per-buffer attributes, and so on. Anything that can be typed into `Ctrl-Z` works in the start-up file.

A minimal `~/.tpurc`:

```
! Show line numbers by default.
SET (LINE_NUMBER, ON);

! Bind Ctrl-G to "what line".
DEFINE_KEY (CTRL_G, "WHAT LINE");

! Define a small editing aid.
```

```
PROCEDURE upper_word
    CHANGE_CASE (CURRENT_LINE, UPPER);
ENDPROCEDURE;
DEFINE_KEY (GOLD_U, "upper_word");
```

7.3 The TPU\$INIT_PROCEDURE hook

DECTPU runs `TPU$INIT_PROCEDURE`, `TPU$LOCAL_INIT`, and (when EVE is loaded) `EVE$INIT` at start-up to give the section file an opportunity to wire up the editor. VX/TPU does **not** invoke any of these automatically; if you compile a procedure with one of those names from `~/tpurc` you must call it yourself (`EXECUTE TPU$INIT_PROCEDURE;`).

8. EVE on top of TPU

The Sector7 build implements the EVE command-line vocabulary directly in C++. `Ctrl-Z` brings up the `*` prompt; `execute_eve_command ( tpu.cpp:3605 )` is the first dispatcher tried, before the TPU interpreter. Anything the EVE dispatcher does not recognise falls through to `execute_command ( tpu.cpp:3485 )` and then to `parser.execute_statement` for raw TPU.

8.1 EVE command vocabulary

Recognised at the `*` prompt (or by `EVE_*`-style script calls if you wrap them in procedures). All commands are case-insensitive.

Command	Action	Source
GET <file>	Open <file> in a new buffer, switch to it	tpu.cpp:3611
INCLUDE <file>	Insert the contents of <file> at the cursor	tpu.cpp:3624
SAVE FILE	Save current buffer	tpu.cpp:3643
WRITE FILE [<name>]	Save (optionally to a new path)	tpu.cpp:3648
WRITE [<name>]	Same	tpu.cpp:3501
SAVE	Same	tpu.cpp:3501
EXIT / QUIT	Leave the editor	tpu.cpp:3499
FIND <text> / SEARCH <pat>	Forward search (literal or pattern)	tpu.cpp:3504
SEARCH_QUIETLY <pat>	Same, no status message	tpu.cpp:3504
REPLACE	Prompt for old/new text	tpu.cpp:3715
TOP	Go to start of buffer	tpu.cpp:3530
BOTTOM / END	Go to end of buffer	tpu.cpp:3532
GO TO <n>	Jump to line n	tpu.cpp:3768
GO TO COLUMN <n> / GO TO COL <n>	Jump to column n	tpu.cpp:3771
WHAT LINE	Report current row of total	tpu.cpp:3795
WHAT BUFFER	Same as SHOW BUFFERS	tpu.cpp:3871
BUFFER <name>	Switch to (or create) a buffer	tpu.cpp:3556
NEXT BUFFER	Rotate to the next buffer	tpu.cpp:3654
PREVIOUS BUFFER	Rotate to the previous buffer	tpu.cpp:3659
BUFFER LIST	Show all buffers (same as SHOW BUFFERS)	tpu.cpp:3866

Command	Action	Source
SHOW BUFFERS	List all buffers	tpu.cpp:4019
SHOW KEYMAPS	List all key maps	tpu.cpp:4027
SHOW KEY_MAP_LISTS	List all key map lists	tpu.cpp:4033
SHOW PROCEDURES	List all user procedures	tpu.cpp:4039
SHOW VARIABLES	List all global variables	tpu.cpp:4045
SHOW DEFAULTS	Per-buffer attributes	tpu.cpp:4071
SHOW SUMMARY	One-screen overview	tpu.cpp:4093
SHOW SYSTEM	Screen / mouse / timer state	
SHOW VERSION	Release banner (V1.08-0 + copyright + date)	
SHOW PASTES	Named paste buffers + unnamed slot preview	
SHOW KEY [<name>]	List bindings or one key	
SELECT / MARK	Drop a selection anchor	
SELECT_RANGE	Materialise the active selection	
COPY / COPY TO <name>	Copy selection to paste; optional named store	
CUT / CUT TO <name>	Cut selection to paste; optional named store	
PASTE / INSERT_HERE	Insert paste; PASTE PREVIOUS rotates kill ring; PASTE <name> / PASTE FROM <name>	
STORE <name> / RECALL <name> / FETCH <name>	Save / load named paste slot without requiring COPY TO	
DELETE	Cut selection or erase one character	tpu.cpp:3702
REMOVE	Cut selection (no fallback)	tpu.cpp:3803
RESTORE	Synonym of PASTE	tpu.cpp:3816
MOVE_TEXT	Cut selection	tpu.cpp:3542
COPY_TEXT "..."	Insert literal text at cursor	tpu.cpp:3544
ERASE_LINE	Delete current line	tpu.cpp:3534
FILL	Fill selected range within current margins	tpu.cpp:3733
SHIFT LEFT [n] / SHIFT RIGHT [n]	Shift every line in the buffer n columns	tpu.cpp:3745
	Upper/lower-case on selection or word	

Command	Action	Source
CHANGE CASE UPPER LOWER / CASE UPPER LOWER		tpu.cpp:3755
COMPILE	Treat the current buffer as TPU source and compile it	tpu.cpp:3558
EXECUTE <name>	Run a previously compiled procedure	tpu.cpp:3560
DEFINE_KEY <key> <text>	Bind a key	tpu.cpp:3563
UNDEFINE_KEY <key>	Remove a binding	tpu.cpp:3582
LOOKUP_KEY <key>	Show what a key does	tpu.cpp:3584
KEY_NAME <key>	Echo the normalised key name	tpu.cpp:3586
LAST_KEY	Show the last keycode	tpu.cpp:3589
READ_KEY	Wait for one keystroke and report	tpu.cpp:3591
CREATE_KEY_MAP <name>	Allocate a new map	tpu.cpp:3565
CREATE_KEY_MAP_LIST <name>	Allocate a new map list (and activate it)	tpu.cpp:3571
ADD_KEY_MAP <list> FIRST LAST <map>	Push a map onto a list	tpu.cpp:3578
REMOVE_KEY_MAP <list> <map>	Remove a map	tpu.cpp:3580
SET INSERT	Switch to INSERT mode	tpu.cpp:3675
SET OVERSTRIKE	Switch to OVERSTRIKE mode	tpu.cpp:3681
SET GOLD KEY <key>	Change the GOLD prefix	tpu.cpp:3664
SET LEFT MARGIN <n>	1-based left margin	tpu.cpp:3821
SET RIGHT MARGIN <n>	1-based right margin	tpu.cpp:3826
SET MARGINS <l> <r>	Both	tpu.cpp:3831
SET TAB STOPS <n>...	Custom tab columns	tpu.cpp:3845
SET LINE NUMBER	Toggle the gutter	tpu.cpp:3839
SET EOB TEXT <text>	Replace the "[End of file]" marker	tpu.cpp:3854
BOX SELECT / BOX CUT / BOX COPY / BOX PASTE / BOX REMOVE	Column-block selection	tpu.cpp:3876
SPAWN [<cmd>] / DCL [<cmd>]	Suspend ncurses, run a shell command, resume	(in the run loop)
HELP [<topic>]	Show help text (topic or full)	tpu.cpp:3597

When the EVE dispatcher does not recognise the command, control falls through and the TPU interpreter tries to evaluate the line as a TPU statement; this is how procedure calls written at the `*` prompt run.

8.2 Kill ring and named pastes

Each buffer has an unnamed linear paste slot (`Buffer::paste_text`) and an optional rectangular `paste_box`. In addition the editor maintains:

- a **kill ring** (up to 32 entries) — `COPY` / `CUT` / selection-`DELETE` push; `PASTE` uses the tip; `PASTE PREVIOUS` rotates older kills;
- **named paste buffers** in `TPUEditor::paste_buffers` — `STORE_PASTE("NAME")` / `RECALL_PASTE("NAME")`, EVE `COPY TO name` / `PASTE name` / `SHOW PASTES`.

The unnamed slot is also filled by `READ_CLIPBOARD` and emptied/consumed by `PASTE` / `INSERT_HERE` / `RESTORE` / `BOX PASTE`.

8.3 EDT\$EMULATE

EVE on VMS exposes `EDT$EMULATE` to give EDT line-mode commands. The Sector7 port has no `EDT$EMULATE` procedure; users wanting EDT-style line editing should use the `vxtools85/edt/edt` binary instead.

9. Journal / Recovery

9.1 Format

By default, `JOURNAL_OPEN` creates a **binary TPJB** journal:

- magic `TPJB` + little-endian `u32` version (1),
- then length-prefixed UTF-8 statement records (`u32` length + bytes).

Legacy **plain-text** journals (one statement per line) remain readable by `RECOVER_BUFFER`, which auto-detects the format from the magic.

Trade-off vs VMS binary keystroke journals: Sector7 records *interpreter* statements (and key bindings' TPU text), not raw keyscan codes.

9.2 Opening and closing

```
JOURNAL_OPEN ("/tmp/my-buffer.tpj");  
SET (JOURNALING, ON);           ! same effect, no path change  
JOURNAL_CLOSE;
```

`JOURNAL_OPEN(path)` truncates the file and sets the per-buffer `journal_path`. `JOURNAL_CLOSE` clears the flag and the path.

Once `journaling` is on, every statement run through `execute_statement` is appended to the journal (see `tpu.cpp:5253-5260`).

9.3 Recovery

```
RECOVER_BUFFER ("/tmp/my-buffer.tpj");
```

Replays the journal line-by-line into the current buffer (or the specified one). Each line is dispatched through `execute_statement`, so the buffer state after recovery matches the state at the point of the last journal write.

There is no automatic recovery at start-up; the user has to invoke `RECOVER_BUFFER` explicitly.

10. Keypad Model

10.1 The default key map

`define_default_keys()` (`tpu.cpp:3040`) installs the `TPU$KEY_MAP` map and a `TPU$KEY_MAP_LIST` list that wraps it. The bindings are:

10.1.1 Movement

Key	Action	Native fn / binding source
Left arrow	<code>MOVE_HORIZONTAL(-1)</code>	<code>tpu.cpp:3049</code>
Right arrow	<code>MOVE_HORIZONTAL(1)</code>	<code>tpu.cpp:3050</code>
Up arrow	<code>MOVE_VERTICAL(-1)</code>	<code>tpu.cpp:3051</code>
Down arrow	<code>MOVE_VERTICAL(1)</code>	<code>tpu.cpp:3052</code>
Home	<code>LINE_BEGIN</code>	<code>tpu.cpp:3058</code>
End	<code>LINE_END</code>	<code>tpu.cpp:3059</code>
PgDn	Forward one screen	<code>tpu.cpp:3090</code>
PgUp	Back one screen	<code>tpu.cpp:3094</code>

10.1.2 Editing

Key	Action	Source
Backspace (127 or <code>KEY_BACKSPACE</code>)	<code>ERASE_CHARACTER(-1)</code>	<code>tpu.cpp:3054-3055</code>
Delete (DC)	<code>ERASE_CHARACTER(1)</code>	<code>tpu.cpp:3056</code>
Enter / Return	<code>SPLIT_LINE</code>	<code>tpu.cpp:3066-3067</code>
Tab	<code>INSERT TAB</code> (uses <code>TAB_STOPS</code>)	<code>tpu.cpp:3068</code>
Insert	Toggle INSERT / OVERSTRIKE	<code>tpu.cpp:3061</code>

10.1.3 Selection / clipboard

Key	Action	Source
Ctrl-V (22)	<code>SELECT</code> (drop mark)	<code>tpu.cpp:3075</code>
Ctrl-W (23)	<code>MOVE_TEXT SELECT_RANGE</code> (cut selection)	<code>tpu.cpp:3076</code>
Ctrl-Y (25)	Paste	<code>tpu.cpp:3077</code>

10.1.4 Windows

Key	Action	Source
Ctrl-O (15)	SPLIT_WINDOW	tpu.cpp:3078
Ctrl-N (14)	NEXT_WINDOW	tpu.cpp:3079

10.1.5 Application

Key	Action	Source
Ctrl-X (24)	EXIT	tpu.cpp:3070
Ctrl-S (19)	WRITE_FILE (save)	tpu.cpp:3071
Ctrl-F (6)	Prompt for search	tpu.cpp:3072
Ctrl-Z (26)	Enter command mode (* prompt)	tpu.cpp:3073

10.1.6 Function keys

Key	Action	Source
F1 / PF1	GOLD prefix (set gold_pending)	tpu.cpp:3081
F2	HELP	tpu.cpp:3086
F3	FIND	tpu.cpp:3087
F4	DELETE	tpu.cpp:3088

10.1.7 GOLD chords

After F1/PF1 (or whichever key holds `gold_key`), the next key is OR'd with 10000 internally and looked up as a GOLD chord. Defaults:

Chord	Action	Source
GOLD T	TOP (go to start of buffer)	tpu.cpp:3099
GOLD E	END (go to end of buffer)	tpu.cpp:3098
GOLD F	FIND (search)	tpu.cpp:3100
GOLD W	WRITE FILE	tpu.cpp:3101
GOLD Q	QUIT	tpu.cpp:3102
GOLD U	UNDO	tpu.cpp:3103
GOLD R	REDO	tpu.cpp:3104

10.2 Key names

`key_name_to_code()` (`tpu.cpp:3440`) accepts these names case insensitively:

Name	Translates to
<code>CTRL_X</code> , <code>CTRL_Z</code> , <code>CTRL_S</code> , <code>CTRL_F</code> , <code>CTRL_V</code> , <code>CTRL_W</code> , <code>CTRL_Y</code>	ASCII control codes
<code>LEFT</code> , <code>RIGHT</code> , <code>UP</code> , <code>DOWN</code>	Arrow keys
<code>DELETE</code>	<code>KEY_DC</code> (delete char)
<code>BACKSPACE</code>	<code>KEY_BACKSPACE</code>
<code>INSERT</code>	<code>KEY_IC</code>
<code>HOME</code>	<code>KEY_HOME</code>
<code>END</code>	<code>KEY_END</code>
<code>NEXT_SCREEN</code> , <code>PAGEDOWN</code>	<code>KEY_NPAGE</code>
<code>PREVIOUS_SCREEN</code> , <code>PAGEUP</code>	<code>KEY_PPAGE</code>
<code>PF1</code> , <code>GOLD</code>	<code>KEY_F(1)</code>
<code>PF2</code> , <code>PF3</code> , <code>PF4</code>	<code>KEY_F(2..4)</code>
<code>F6 ... F12</code>	<code>KEY_F(6..12)</code>
<code>GOLD_<key></code>	<code><key> + 10000</code>
any single ASCII character	the character itself

Examples:

```
DEFINE_KEY GOLD_T    "TOP"
DEFINE_KEY F7        "SHOW BUFFERS"
DEFINE_KEY CTRL_G    "WHAT LINE"
```

10.3 GOLD prefix lifecycle

1. Pressing the GOLD key (`gold_key`, default F1) sets `gold_pending = true` and shows `GOLD` on the message line.
2. The next keypress is **transformed**: `key + 10000` is the lookup key for the second keystroke (`tpu.cpp:3081`).
3. The flag is cleared regardless of whether a binding was found.

To change which physical key acts as GOLD:

```
SET GOLD KEY F12
```

or programmatically:

```
gold_key := 277;      ! some keycode
```

10.4 Custom bindings

```
CREATE_KEY_MAP    my_map
CREATE_KEY_MAP_LIST my_list
ADD_KEY_MAP      my_list    LAST    my_map
ADD_KEY_MAP      my_list    LAST    TPU$KEY_MAP      ! fall through
DEFINE_KEY       F6        "BUFFER notes"
SET (KEY_MAP_LIST, "my_list")
```

The active list is searched in order; the first map that has a binding for the keystroke wins. Per-buffer maps override globals: `SET (KEY_MAP_LIST, "my_list", buffer)` binds a list to one buffer.

11. Pattern Language

11.1 BNF sketch

```
pattern_expr ::= pattern_term ('+' pattern_term | '&' pattern_term | '|'
pattern_term) *
pattern_term ::= ANCHOR | UNANCHOR | LINE_BEGIN | LINE_END | PAGE_BREAK
               | REMAIN
               | ANY    '(' string ')'
               | NOTANY '(' string ')'
               | ARB    '(' integer ')'
               | SCAN   '(' string ')'
               | SCANL  '(' string ')'
               | SPAN   '(' string ')'
               | SPANL  '(' string ')'
               | string-literal
               | '(' pattern_expr ')'
```

Anchored patterns must match at the cursor; unanchored patterns slide forward.

11.2 Operator semantics

- **A + B** - concatenation; if **A** is exhausted without consuming the needed input, the matcher backs up and re-tries **A** with an alternative shape.
- **A & B** - concatenation but **B** is marked `no_backtrack`; once matching **B** fails, the whole pattern fails immediately.
- **A | B** - alternation; the matcher tries **A**, then **B**.

11.3 Examples

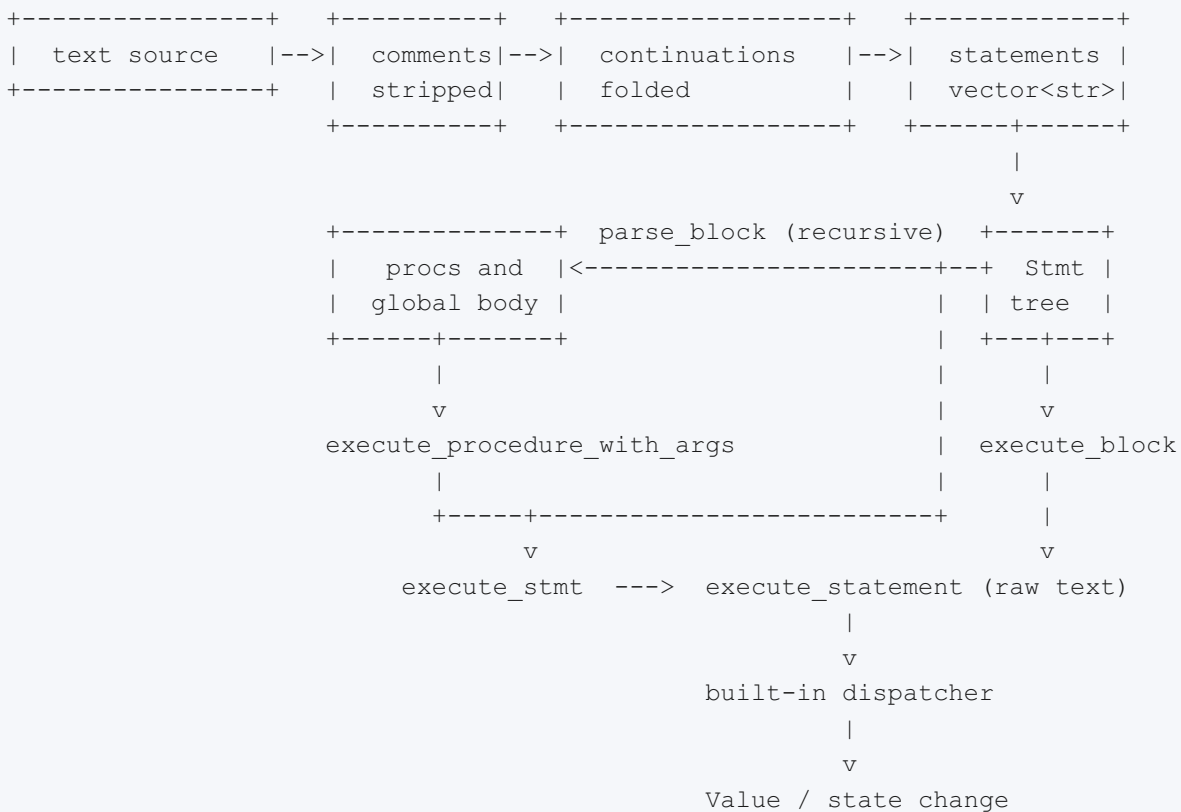
Goal	Pattern
Lines starting with !	<code>LINE_BEGIN + "!"</code>
Whole ! - comment lines	<code>LINE_BEGIN + "!" + REMAIN</code>
A C-style identifier at the cursor	<code>ANCHOR + ANY("abcdefghijklmnopqrstuvwxyz_") + SPANL("abcdefghijklmnopqrstuvwxyz0123456789_")</code>
Exactly 4 arbitrary characters anywhere	<code>ARB(4)</code>
Skip to the next dot	<code>SCAN(". ")</code>
Maximal run of digits	<code>SPAN("0123456789")</code>

12. Compile Model - How `tpu.cpp` Parses and Runs

12.1 The interpreter is hand-rolled

There is no `flex` / `bison`, no AST traversal of token streams. The parser works at the **statement string** level: source text is split into statements by `;` / `\n` and each statement is dispatched at runtime by name. The result is unusually simple but means there is no intermediate byte code - re-running a procedure parses every statement again.

12.2 The pipeline



Source pointers:

- `compile_text` - `tpu.cpp:2512`
- `parse_block` - `tpu.cpp:2291`
- `execute_block` - `tpu.cpp:6030`
- `execute_stmt` - `tpu.cpp:6064`
- `execute_statement` - `tpu.cpp:5212`
- `execute_procedure_with_args` - `tpu.cpp:2571`
- Expression evaluator - `eval_or` -> `eval_and` -> `eval_compare` -> `eval_add` -> `eval_mul` -> `eval_unary` -> `eval_primary` (see `MiniTPUParser` class definition, `tpu.cpp:2073-2143`).

12.3 Lexer / scanner

There is no token type. The "lexer" is:

- `strip_comments` (tpu.cpp:2177) - replace `! ... newline` with `\n`.
- `join_continuations` (tpu.cpp:2197) - fold `-\n` into nothing.
- `split_statements` (tpu.cpp:2220) - split at `;` or `\n` while honouring `"..."` quoting and nested `(...)`.

The expression evaluator scans characters one at a time using the `MiniTPUParser::skip_ws`, `match_word`, and `split_top_args` helpers (tpu.cpp:2145-2150, tpu.cpp:5190).

12.4 Parser

`parse_block` (tpu.cpp:2291) is a hand-rolled recursive-descent recogniser that consumes statement strings until it sees a terminator keyword (`ENDIF`, `ELSE`, `ENDLOOP`, `ENDPROCEDURE`, `ENDON_ERROR`). For every statement it produces zero or more `Stmt` nodes:

- `LOCAL` / `CONSTANT` / `VARIABLE` -> zero or more `ASSIGN`.
- `IF` -> `Stmt{Kind::IF, text=cond, body=..., else_body=...}`.
- `LOOP` -> `Stmt{Kind::LOOP, body=...}`.
- `EXITIF` -> `Stmt{Kind::EXITIF, text=cond}`.
- `RETURN` / `ABORT` / `BREAK` -> flow-only `Stmt`.
- `ON_ERROR` -> `Stmt{Kind::ON_ERROR, body=handler}`.
- Anything else with `:=` -> `Stmt{Kind::ASSIGN, text=lhs, rhs=expr}`.
- Anything else -> `Stmt{Kind::CALL, text=original}` (dispatched later).

12.5 Expression evaluator

Recursive-descent with one function per precedence layer (`eval_or` -> `eval_and` -> `eval_compare` -> `eval_add` -> `eval_mul` -> `eval_unary` -> `eval_primary`). The expression text is passed by `(const string&, size_t& i)` and each function advances `i` as it consumes input.

`eval_primary` handles literals, identifier lookup, function calls, parenthesised sub-expressions, and array subscripts.

12.6 Built-in dispatcher

The dispatcher is a long sequence of `if (name == "NAME") { ... }` clauses (about 100 of them in each of `execute_statement` and `eval_function`). The grouping is thematic - cursor motion first, then text editing, then mark/range, then search, etc. - mirroring the section structure of this manual.

This design choice trades a small amount of run-time overhead for **diff-friendly source**: adding a new built-in is one self-contained block. There is no registration table; the dispatcher *is* the table.

12.7 Memory model

Values are reference-counted by `std::shared_ptr` for the collection-shaped payloads (`pattern`, `range`, `marker`, `array`, `program`, `learn_seq`, `process`). Buffers, windows, and key maps are owned by `TPUEditor` via `std::unique_ptr`; `Value::BUFFER` carries a non-owning raw pointer.

Undo / redo snapshots are full copies of the buffer's line vector. A per-buffer cap (`undo_limit`, default 200) bounds memory.

12.8 Top-level C++ layout

Section in <code>tpu.cpp</code>	Range	Purpose
Platform shims	54-167	ncurses/PDCurses, signal handler
Marker / Range	169-274	Position values
TPUPattern	276-529	Pattern token + matcher
Buffer	531-1582	Text, cursor, undo, video
TPUProcess	1583-1785	External process + pipe
TPUWindow	1786-1814	Viewport
KeyBinding / KeyMap	1815-1907	Keymap layer
Stmt / Value / Procedure	1908-2024	Compiler data model
MiniTPUParser	2073-2614	Lexer + parser + interpreter
TPUEditor	2645-...	Screen + run loop + command line
FAO + expressions	6500-7000	FAO, GET_INFO, pattern primitives
VX_TTY redirection	7010-7058	Newterm() bridge
Splash screen	7072-7197	Optional banner
<code>main</code>	7212-7282	<code>argv</code> , <code>--test</code> , <code>\$TPU_STARTUP</code> , <code>run()</code>

13. Known Divergences from VAX/VMS TPU

VX/TPU implements a faithful **character-cell subset** of DECTPU. OpenVMS cross-checks on host `t86` (x86_64 V9.2-3) confirm `GET_INFO(SYSTEM, "version")=3 / "update"=2` and `WRITE_FILE(buffer, filespec) (%TPU-S-FILEOUT)`. Sector7 reports its own `TPU_SYSTEM_VERSION / TPU_SYSTEM_UPDATE` (currently 1 / 0) through the same SYSTEM selectors.

13.1 Policy-deferred (not in the close loop)

- **RMS record formats** — POSIX approximations, not full Files-11 RMS. (**OpenVMS ;n file versioning is now implemented** — a bare name opens the highest version and each save writes the next ;n; enabled by `TPU/VERSIONING` in the VX/RT config, `SET VERSIONING ON` in `TPU.INI` or interactively, or `S7SECURITY.INI versioning=.` See the EDT reference §13.6 for the shared behaviour.)
- **DECwindows Motif widgets** — `CREATE_WIDGET / MANAGE_WIDGET / ...` accepted as no-ops.
- **CONVERT / GET_DEFAULT / global selection** — accepted no-ops.
- **VMS bytecode** `.TPU$SECTION` — Sector7 uses portable `#VXSECT` source sections instead (see §7.1).
- **VMS keystroke** binary journal layout — Sector7 uses TPJB framed statement journals (see §9.1).

13.1b Closed formerly-deferred items

- **Section SAVE / RESTORE / EXPAND + /SECTION=** — portable VXSECT.
- **Binary journal** (`JOURNAL_OPEN / RECOVER_BUFFER`) — TPJB format.
- **Callable API** — `tpu_api.h / libtpu` with `TPU_Initialize`, `TPU_ExecuteCommand`, `TPU_Cleanup`, and `TPU$*` symbol aliases.

13.2 Behavioural notes (host / subset)

- **BUFFER("name")** is invalid on DECTPU (`TPU$_NOTARRAY`); use `CREATE_BUFFER / GET_BUFFER` return values. Sector7 still accepts a buffer-name form for convenience.
- **GET_INFO(0, "VERSION")** is a Sector7 extension (string `V1.08-0`). OpenVMS rejects integer subject + `"VERSION"`. Prefer `GET_INFO(SYSTEM, "version"|"update")` for DEC-compatible probes.
- **DELETE(buffer)** and **undo Cleanup** — deleting a buffer that was snapshotted for statement undo must clear `snap_buf` / wipe live bindings before freeing; otherwise glibc may abort on exit (corrupted size vs. prev_size). Fixed in V1.01-0.
- **WRITE_FILE("path")** alone is invalid on DECTPU; use `WRITE_FILE(buffer [, filespec])`. Host string form retained for EVE.
- **SPAWN** runs the host shell (`system()` / `/bin/sh`), not DCL.
- **Pattern expressions** are cached (≤ 256 entries) after tokenisation; they are not compiled to a DEC-style state machine.
- **Status / prompt layout** — global status row sits above the bottom command/prompt line (character-cell prompt-on-bottom).

- **Buffer name is the file path** when opened from the command line (not always `MAIN` with a separate file attribute).
- **No multi-key chords beyond GOLD** — `SHIFT_KEY` is a second GOLD-style prefix only.
- `LBR_*` / `LIB_*` VMS library bindings are not implemented.

13.2b Arrays (DEC surface implemented)

- `CREATE_ARRAY` / `CREATE_ARRAY(n [, start])` — dynamic + static portions.
- Element walk: `GET_INFO(arr, "FIRST"|"NEXT"|"LAST"|"PREVIOUS"|"CURRENT")`.
- Bounds: `"LOW_INDEX"` / `"HIGH_INDEX"`; soft size `"SIZE"` / `"LENGTH"`.
- Clear dynamic slot: `arr{k} := UNSPECIFIED` (or `TPU$K_UNSPECIFIED`).
- `DELETE(arr)` marks the array dead and sets referencing variables to `UNSPECIFIED`.
- Global list: `GET_INFO(ARRAY, "FIRST"|"NEXT"|...)`.
- Integer indexes stay typed; other indexes stringify. `$` allowed in identifiers.

13.3 Partial forms still differing from full DEC

Built-in / area	Notes
<code>SCROLL</code> / <code>SHIFT</code>	Window/buffer + integer amount; no fractional DEC variants
<code>CALL_USER</code>	POSIX <code>dlopen</code> bridge; not VMS <code>LIB\$</code> calling standard
<code>KEY_NAME</code>	Partial — normalised echo / lookup
App. B messages	~325 symbols catalogued; OCR-cleaned from Reference Manual

13.4 Sector7 additions DEC TPU does not have

- `--test` headless mode with `ASSERT_*` / `TEST "label"`.
- `HELP_TEXT(topic)` returns help text as a value.
- `BOX SELECT` / `CUT` / `COPY` / `PASTE` / `REMOVE` column-block selection.
- `WRITE_CLIPBOARD` / `READ_CLIPBOARD` via host clipboard tools.
- `%X`, `%O`, `%B`, `%D` numeric literal prefixes.
- `GET_INFO(0, "VERSION"|"PLATFORM"|"RELEASE_DATE")`.
- `STORE_PASTE` / `RECALL_PASTE` and EVE named-paste vocabulary.
- Kill ring (`PASTE PREVIOUS`).
- `SHOW SUMMARY` / `SHOW SYSTEM` / `SHOW VARIABLES` / `SHOW VERSION` / `SHOW PASTES`.
- Release identity macros: `TPU_VERSION_*`, `TPU_RELEASE_DATE`, `TPU_SYSTEM_VERSION` / `TPU_SYSTEM_UPDATE`.

14. File Reference

14.1 The source tree

```
/Users/jonpower/s7tools/vxtools85/tpu/
+-- makefile                Sector7 wrapper makefile
+-- run_tests.sh            Build then run regression suite
+-- tpu.cpp                 The whole editor (~8,000 lines)
+-- tpu_messages_appendix_b.inc  Appendix B MESSAGE_TEXT catalog
+-- tpu.exe                 Compiled binary
+-- tests/
|   +-- tpu_tests.tpu       Core regression (--test)
|   +-- tpu_complete.tpu   Extended regression
|   +-- tpu_coverage.tpu   Feature coverage (C01-C12)
|   `-- README.md / COVERAGE.md
+-- t86/
|   +-- run_probe.sh / run_crosscheck.sh
|   `-- *.com              OpenVMS EDIT/TPU probes
+-- TPU_XCODE/             Xcode project for IDE work
`-- docs/
    +-- TPU_REFERENCE.md / .pdf    This manual
    +-- TPU_CHANGELOG.md / .pdf    Release notes
    +-- TPU_MESSAGES_APPENDIX_B.md  Extracted facility texts
    +-- DEC_TPU_Guide_V7.3.pdf
    +-- DEC_TPU_Reference_Manual.pdf / .txt
    +-- extract_appendix_b_messages.py
    `-- build_pdf.py            Markdown -> PDF builder
```

14.2 Key entry points

Area	Purpose
main	argv (--test , /COMMAND= , /SECTION=), splash, run loop
vx_redirect_tty	\$VX_TTY newterm() bridge
MiniTPUParser	Compile + execute TPU statements
TPUEditor	Screen, EVE command line, kill ring, pastes
tpu_lookup_message	Appendix B MESSAGE_TEXT / MESSAGE
GET_INFO dispatch	Buffer / range / marker / window / SYSTEM
t86/*.sh	OpenVMS cross-check (shell Task + ["all"])

14.3 Test harness

`tests/tpu_tests.tpu` exercises core language, builtins, and recent fidelity work via `TEST "section" / ASSERT_*`. Run headlessly with:

```
./run_tests.sh
# or: ./tpu.exe --test tests/tpu_tests.tpu
```

As of V1.01-0 this reports **104 passed, 0 failed** on `tpu_tests.tpu` (plus complete/coverage in `./run_tests.sh`). Exit code is 0 when all asserts pass, 1 otherwise. The harness uses the same TPU dispatcher as interactive mode.

14.4 Build

POSIX (Linux / macOS):

```
g++ -std=c++17 -Wall -Wextra -O2 -I../s7security tpu.cpp \
    ../s7security/s7_sha256.cpp ../s7security/s7_security.cpp -lncurses -o
tpu.exe          # macOS
g++ -std=c++17 -Wall -Wextra -O2 -I../s7security tpu.cpp \
    ../s7security/s7_sha256.cpp ../s7security/s7_security.cpp -lncurses -ldl -o
tpu.exe          # Linux
```

Windows (MSVC + PDCurses):

```
cl /std:c++17 /EHsc /I..\s7security tpu.cpp ..\s7security\s7_sha256.cpp ..
\s7security\s7_security.cpp ^
/I C:\pdcurses C:\pdcurses\wincon\pdcurses.lib
```

The repo's `run_tests.sh` builds with `-O0` and immediately runs the regression suite. On Linux it links `-lncurses -ldl` (RHEL8 requires `-ldl` for `dlclose`). The `makefile` builds with `-g`, links `-lncurses`, and installs into `$(BINDIR)/tpu.exe`.

14.5 Compile-time options

Macro	Effect
<code>TPU_NO_SPLASH</code>	Omit the start-up splash for a single run (no persist)
<code>TPU_SPLASH_SCREEN</code>	(Inverse) Compile the splash screen in

The `S7SECURITY` file is **optional** at runtime (see §2.3) — TPU runs with or without it; it only governs the one-time splash. Build links `../s7security/s7_sha256.cpp` and `../s7security/s7_security.cpp` statically (no OpenSSL).

15. End Notes

15.1 Copyright

The `tpu.cpp` source carries (c) Sector7 USA LLC 2026. All Rights Reserved. The original DEC TPU reference (`tpu.docx`, `tpu.pdf`) is VSI's documentation of DECTPU; it is reproduced here as reference material under VSI's `docinfo@vmssoftware.com` distribution terms.

15.2 Build provenance

- Source: `/Users/jonpower/s7tools/vxtools85/editors/tpu/tpu.cpp` (~8,800 lines, V1.08-0).
- Built and tested on macOS arm64 (`tpu_osx_arm86.exe`) and RHEL8 `rh8x` (`tpu_linux_x86.exe`) with `g++ -std=c++17`.
- Regression: `./run_tests.sh` → **104 / 35 / 134** passed + callable (2026-07-15). Linux headless stress 10× clean after DELETE UAF fix.
- OpenVMS cross-check: `t86 DECTPU version=3 update=2; WRITE_FILE FILEOUT via t86/run_crosscheck.sh`.

15.3 Authorship and contact

Maintained inside the `vxtools85` source tree at Sector7 USA LLC. For DEC semantics see `docs/DEC_TPU_Reference_Manual.pdf` / Guide V7.3; for release notes see `docs/TPU_CHANGELOG.md`.

15.4 Document history

- 2026-05 — Initial VX/TPU C++ port reference (EVE hard-wired, `--test`, splash, `VX_TTY`).
- 2026-07-15 — V1.00-0 fidelity pass: SYSTEM GET_INFO, WRITE_FILE DEC form, Appendix B MESSAGE_TEXT catalog, named pastes / kill ring, ATTACH I/O, FREE_CURSOR, CLI / `COMMAND=` / `SECTION=`, `t86` cross-check results; divergences section rewritten around policy deferrals.
- 2026-07-15 — V1.01-0: DELETE-buffer undo Cleanup use-after-free fix (Linux glibc aborts); `-ldl` on Linux test/regression scripts; regression counts updated (104 / 35 / 134).

End of VX/TPU Reference Manual.